

Stealing Ethereum

How L2 architecture becomes *enterprise infrastructure*

Three series translating Ethereum's most-tested architectural patterns into decisions a technical leader can make tomorrow. **Critical, sourced, and reviewable** — every claim is cited, every episode includes a Counterarguments & Limitations section, every analogy is named as an analogy with its scope marked.

ARCHITECTURE TRANSFER

DECISION ECONOMICS

INDUSTRIAL SYSTEMS

SERIES

3 — Steal · ROI · Industrial

EPISODES

15 — flagship in long-form

REFERENCES

80 — IEEE numbered

TABLE OF CONTENTS

Contents

Editorial Methodology

Series 1 — Steal the Architecture

Episode 01 — Steal Ethereum's Layering Model

Episode 02 — Steal the Rollup Model for Enterprise AI

Episode 03 — Steal the Sequencer

Episode 04 — Steal Data Availability

Episode 05 — Steal Settlement

Series 2 — Architecture ROI

Episode 01 — When Is an L2 Actually Cheaper?

Episode 02 — The Sequencer Economics

Episode 03 — ZK: When the Proof Is Worth Paying For

Episode 04 — The Cost of Verification

Episode 05 — Build vs Buy vs Rollup

Series 3 — Crypto Architecture → Industrial Architecture

Episode 01 — What Ethereum Teaches Us About Factories

Episode 02 — What Rollups Teach Us About Logistics

Episode 03 — What Sequencers Teach Us About Mobility

Episode 04 — What Data Availability Teaches Us About Supply Chains

3

7

7

11

15

19

22

26

26

28

31

33

35

38

38

41

43

45

Episode 05 — What ZK Teaches Us About Industrial Compliance47

Cross-Series Synthesis49

Bibliography51

HOW THIS DOCUMENT IS CONSTRUCTED, AND HOW TO READ IT

Editorial Methodology

Thesis. *This document is not an educational content farm. Every section is designed to move a technical leader from understanding Ethereum L2 architecture to making a better business decision with that understanding — and every claim is sourced, contested, and explicit about where the analogy it relies on breaks.*

A. The monetization filter

We hold ourselves to a single editorial test before publishing any episode: does the reader end the section able to make a materially better business, architecture, or risk decision than they were before they read it? If the only thing an episode transfers is vocabulary — “now you know what a sequencer is” — we have failed. Vocabulary is freely available. What is scarce, and what technical buyers will pay for, is the transfer of a mental model that changes which decisions get made and how [1, 2].

Concretely, we want every series to move the reader from “I understand Ethereum L2s” to “I can use this thinking to make a better business or architecture decision.” That move has four sub-moves we will return to throughout the document: a transfer of pattern (the architecture idea, abstracted from its native context), a transfer of economics (when does this pattern actually make money or save money), a transfer of risk (which failure modes transfer and which do not), and a transfer of decision (which concrete choice does this enable tomorrow). If we cannot answer all four, the section is not yet ready.

This is a deliberately narrow filter. It rules out the bulk of blockchain-related content that exists in the public sphere: explainer articles that translate technical terms without transferring decision power, ranking articles that compare L2s as if they were consumer products, and most opinion writing about “the future of Web3” which has no decision-theoretic content at all. We make no apology for the narrowness. The audience we are writing for — CTOs, technical founders, architects, innovation leaders, and the board-level decision makers they report to — does not have time for content that does not pay back the minute it costs to read [3].

B. What “feedback-reviewable” means in this document

We use the phrase “critical-reviewable” deliberately, and we want to be explicit about what we mean by it because the phrase is doing real work. Three properties define it. First, every non-trivial empirical claim is sourced. We cite Ethereum Improvement Proposals, rollup specifications, peer-reviewed distributed systems literature, and the small set of papers that have shaped how the field thinks about sequencer economics, MEV, and proof systems [4, 5]. Where we cite a primary source, we link to the canonical version. Where we cite a secondary source, we name it as such.

Second, every episode includes a named section titled “Counterarguments & Limitations” that is not decorative. The section exists to identify, in our own voice, the places where the Ethereum-to-enterprise

analogy we are proposing breaks. We do this because the most useful critiques of any architecture transfer are the ones the author raises themselves before they are used against the proposal in a serious review. If a reader disagrees with our analysis, we want the disagreement to be about whether we have weighted the limitations correctly — not about whether we have hidden them. Steelman criticism, in our usage, means we adopt the strongest available counter-position before responding to it [6].

Third, where we use a metaphor, we name it as a metaphor and we specify its scope. The most common failure mode in architecture writing is to slide between analogy and identity — to start with “this is like a rollup” and end with “this is a rollup,” without ever saying where the analogy stopped holding. We mark the slide explicitly. Where the analogy breaks, we say so in the body of the section, not in a footnote.

This is a higher bar than most published architecture analysis clears. It is the bar we want our own work to be measured against, and it is the bar we recommend our readers impose on any architecture proposal that arrives at their desk claiming lineage from a novel system pattern.

C. Editorial voice

We write in the editorial plural. “We observe,” “we argue,” “we are skeptical.” This is not the royal plural — it is the plural of an editorial voice that wants to be identified as a position taken by the publication, not as the personal opinion of a single author. The register is closer to a Forrester or Gartner technical brief than to an academic paper: precise, hedged where hedging is warranted, declarative where the evidence supports it, and unwilling to manufacture false balance between positions of unequal merit [7, 8].

We avoid two registers that are common in adjacent fields. We do not write in the manifesto voice of crypto-native publications, because that voice tends to trade rigor for intensity and we cannot afford that trade. We also do not write in the passive, citation-per-claim voice of academic papers, because that voice is unreadably heavy for the audience we are addressing and tends to smuggle in uncontested assumptions under the cover of neutral-sounding prose. The middle ground we have chosen is harder to write in than either extreme, but it is the register that actually gets read.

D. Reference conventions

Citations are numbered inline using square brackets — [1], [2], [3, 7] — in the IEEE style. The full bibliography appears at the end of the document, ordered by first citation. Where a source is canonical (an EIP, a published paper, a standards document), we cite the canonical version. Where a source is industry documentation (Optimism's docs, Arbitrum's specs, L2Beat's risk framework), we cite the current published version and we note that documentation evolves [9, 10].

The reference corpus spans three categories. The first is Ethereum protocol and L2 primary sources: EIPs, rollup specifications, Vitalik Buterin's architectural writings, the L2Beat risk framework, and the technical documentation of the major L2 teams. The second is the distributed systems and enterprise architecture literature: Lamport and Schneider on fault-tolerant distributed systems, Hohpe and Woolf on enterprise integration patterns, Evans and Vernon on domain-driven design, Fowler on enterprise application architecture, and the NIST zero trust architecture specification. The third is the decision economics literature: build-versus-buy frameworks, real options analysis, capital expenditure modeling,

and the small but growing academic literature on MEV and transaction fee mechanism design.

We have deliberately not included two reference categories that are adjacent and that readers might expect. We do not cite industrial compliance standards (ISO 21434, ISA-95, GS1 EPCIS, NIST AI RMF) in the main corpus — those appear in Series 3 where they are directly load-bearing, but we want the core argument to stand on its own without appeal to a particular regime's audit framework. We also do not cite the trust infrastructure literature (zero trust architecture, verifiable credentials) in the main corpus — those are referenced where they are directly relevant, primarily in the methodology and cross-series synthesis, but we did not want the editorial voice to drift into a standards-document register.

E. The Content → Money architecture

The three series in this document are designed to interlock as a single commercial funnel. The funnel is not a sales funnel in the Marketing 101 sense — it is an architecture funnel in the sense that each stage transfers a specific kind of capability, and the capability transferred at each stage is the prerequisite for the buyer to want the next stage. Figure 0.1 shows the structure.

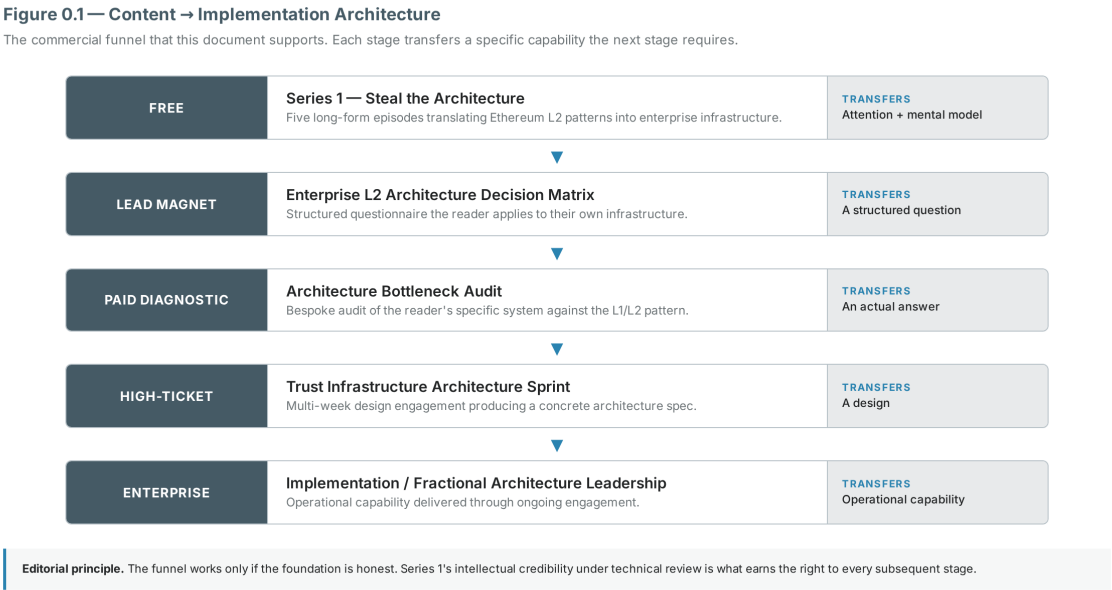


Figure 0.1 — The content-to-implementation architecture. Each stage transfers a specific capability that the next stage requires.

Read the figure from top to bottom. The free content layer transfers attention and a mental model. The lead magnet layer — in our concrete plan, an Enterprise L2 Architecture Decision Matrix — transfers a structured question the reader can apply to their own infrastructure. The paid diagnostic, the Architecture Bottleneck Audit, transfers an actual answer to that question for their specific system. The high-ticket Trust Infrastructure Architecture Sprint transfers a design. The enterprise implementation layer transfers operational capability [11].

This structure is important because it constrains what each series needs to do. Series 1 (Steal the Architecture) must succeed as content first; if it does not generate attention and transfer a mental model,

the rest of the funnel is unreachable. Series 2 (Architecture ROI) must succeed as a qualification mechanism; if it does not help a reader identify whether the architecture is economically viable for them, the diagnostic layer has no entry point. Series 3 (Industrial Architecture) must succeed as translation; if a logistics CTO cannot see themselves in the episodes, the implementation layer never gets scoped.

F. How to read the three series

Series 1 (Steal the Architecture) is the flagship. It is written in long-form — each episode is roughly 3,000 words of prose, with embedded diagrams, named counterarguments, and a reference list of five to six sources per episode. The flagship status reflects its commercial role: it is the body of work that has to be defensible under technical review, because it is what gets circulated and what generates the inbound signal that feeds the rest of the funnel. We invest asymmetrically in Series 1 because the cost of a weak flagship is far higher than the cost of a weak supporting series.

Series 2 (Architecture ROI) and Series 3 (Crypto Architecture → Industrial Architecture) are written as structured blueprints. Each episode is roughly 1,800 words and is organized around a thesis, five named mechanisms, a set of critical caveats, a counterarguments section, and three to five references. The blueprint depth is deliberate: these series are qualification tools, not exhaustive treatises. They need to transfer enough architecture thinking for a technical buyer to recognize whether the pattern is worth a deeper conversation, without pretending to be the deeper conversation itself.

The three series interlock as Attention → Decision → Translation. Series 1 generates the attention and the mental model. Series 2 turns the attention into a structured decision the reader can apply to their own context. Series 3 translates the decision into the operational language of a specific industry — manufacturing, logistics, mobility, supply chain, industrial compliance. A reader who completes all three series has, in effect, completed a self-paced architecture seminar on the subject, and is qualified to enter the paid diagnostic layer with a real problem to be audited.

We close the methodology with one editorial commitment. Where the field's enthusiasm outruns its evidence — and in blockchain-adjacent architecture, this happens often — we will say so. Where the architecture is genuinely powerful and the enterprise translation is genuinely viable, we will say that too, with equal force. The credibility of the second claim depends entirely on our willingness to make the first.

THE FLAGSHIP. FIVE LONG-FORM EPISODES THAT TRANSLATE ETHEREUM'S ARCHITECTURAL PATTERNS INTO REUSABLE ENTERPRISE INFRASTRUCTURE.

Series 1 — Steal the Architecture

Thesis. *You do not need to build a blockchain to use blockchain architecture. Each episode steals one pattern from Ethereum's L2 stack and translates it into a design move available to any technical leader.*

The positioning of this series is the positioning of the document as a whole: we are not teaching Ethereum. We are using Ethereum's architecture as a body of solved problems — a tested catalogue of design moves for trust, sequencing, evidence, settlement, and verification — and we are stealing those moves for use in systems that have nothing to do with cryptocurrency. That is a strong intellectual property position, and it is also the position that earns the right to the commercial funnel that follows in Series 2 and Series 3.

Each episode is structured identically. We open with a one-sentence thesis. We explain the mechanism: how the pattern works inside Ethereum, with citations to the canonical sources. We translate the mechanism: how the same design move can be stolen and applied to an enterprise architecture problem. We then turn to critical analysis — where the analogy holds and where it does not — and we close with a dedicated Counterarguments & Limitations section that names, in our own voice, the failure modes a serious reviewer would raise.

The series is the longest in this document because it is the one that has to survive technical review. If Series 1 is shallow, the credibility that Series 2 and Series 3 depend on evaporates. We have invested the writing time accordingly.

FROM L1/L2 SEPARATION TO CORE SYSTEM / SPECIALIZED EXECUTION LAYER

Episode 01 — Steal Ethereum's Layering Model

Thesis. *Ethereum's most replicable architectural idea is not the blockchain. It is the deliberate separation of an expensive, slow, high-trust settlement core from cheap, fast, specialized execution layers — and that separation is a design move available to almost any complex digital system, with or without cryptographic primitives.*

Mechanism

Ethereum's L1 is, by deliberate design, slow. The chain processes roughly 15 transactions per second on the base layer under normal conditions, and the gas market clears through an auction that prices compute,

storage, and data publication at marginal cost [1, 2]. This is not a flaw to be optimized away. It is an architectural choice: the base layer is the trust root, and the trust root must be conservatively governed, widely replicated, and expensive to attack. Making the trust root cheap to write to would compromise its role as the trust root.

Layer 2 rollups — Arbitrum, Optimism, Starknet, zkSync, the zkEVM family — solve the throughput problem by relocating execution off the trust root. A rollup executes transactions locally, batches state transitions, and posts a compressed summary of those transitions to L1 as calldata or blobs [3, 4]. L1 does not execute the rollup's transactions; it settles them, anchoring the rollup's state to a trust root whose security it already guarantees. The rollup inherits L1's trust guarantees without paying L1's per-transaction execution cost.

The separation is load-bearing because the two layers have different optimization targets. L1 optimizes for censorship resistance, decentralization of validators, and economic security of the trust root — properties that are expensive and that scale poorly. L2 optimizes for execution throughput, low latency, and application-specific feature velocity — properties that are cheap and that scale well. Forcing both into a single layer produces the worst of both: slow, expensive, and feature-poor. Separating them lets each layer be what it should be.

The rollup-centric roadmap that Ethereum committed to in late 2020 — codified in Buterin's rollup-centric roadmap post and operationalized through EIP-4844 proto-danksharding [5, 6] — made the separation a first-class architectural principle rather than a workaround. Blob storage on L1 exists specifically to give rollups a cheap publication layer without forcing L1 validators to execute the rollup's transactions. The entire data-availability layer of the stack exists to serve this separation. This is what an architectural commitment to layering looks like when it is taken seriously.

Architecture Translation

The steal is to recognize that the L1/L2 pattern is a specialization of a much more general move: separate the trust-critical, slow-moving, conservatively-governed core of your system from the execution-heavy, fast-moving, application-specific periphery, and design the interface between them deliberately. Most enterprise systems already have this shape, in some form — a core transactional system backed by an ERP, surrounded by application services that execute business logic. What they typically lack is the deliberate interface design that makes the separation a load-bearing architectural feature rather than an accident of history.

Figure 1.1 — L1/L2 Layering Pattern → Core / Specialized Execution Layer

The left column shows Ethereum's native L1/L2 separation; the right shows the enterprise translation.

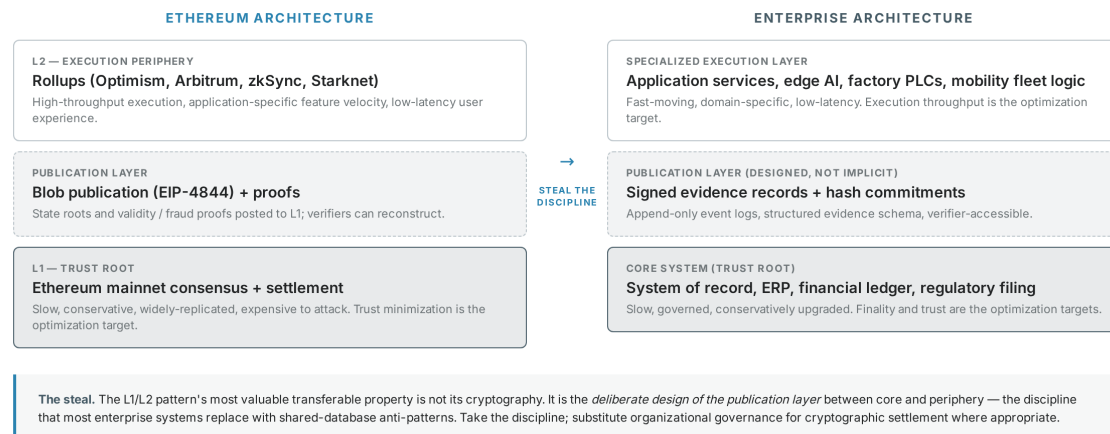


Figure 1.1 — The L1/L2 layering pattern, redrawn as the Core / Specialized Execution Layer pattern. The trust-critical core remains slow, governed, and conservative; execution moves to specialized layers that inherit trust from the core without paying its execution cost.

Concretely, the translation asks four questions of any enterprise system. First: what is the trust root — the component whose correctness must be guaranteed for the rest of the system to be meaningful? In a manufacturing execution system, this might be the master production record and the calibration chain it depends on. In a financial settlement system, it is the ledger of record. In an AI governance system, it is the model card, the training data provenance, and the audit trail of decisions taken.

Second: what is the execution periphery — the components that produce derived state, locally, and at high throughput? These are the systems that need feature velocity, low latency, and domain-specific optimization. They are also the systems that should not be permitted to corrupt the trust root directly.

Third: what is the publication layer — the mechanism by which the periphery commits its state changes back to the trust root, in a form the trust root can verify? Ethereum's answer is blob publication plus fraud or validity proofs. The enterprise equivalent is some combination of append-only event logs, signed evidence records, and verifier-side reconciliation. The key design choice is that the periphery commits evidence, not raw state, to the core.

Fourth: what is the verification strategy — optimistic or validity-based? Ethereum supports both: optimistic rollups assume state is correct unless challenged within a window, while validity rollups require a proof before state is accepted [7, 8]. Enterprise systems implicitly use optimistic verification for the vast majority of their flows, and validity verification (audit, regulatory attestation, third-party certification) only for the flows where the stakes justify the cost. The architectural move is to make this trade explicit, per-flow, rather than implicit and uniform.

Applications of this pattern cut across domains. In manufacturing, the ISA-95 model already separates enterprise planning (level 4) from plant operations (levels 2-3) [9]. What the L1/L2 pattern adds is the publication-layer and verification-strategy design discipline that ISA-95 leaves underspecified. In logistics, the same pattern applies to consignment state: the core settles ownership transfer, the periphery tracks in-transit state. In AI infrastructure, the core settles model provenance and decision audit, the

periphery executes inferences locally. In payments, the core settles value, the periphery executes ledger entries. In mobility, the core settles trip state and authorization, the periphery executes routing and local control. The same architectural skeleton serves all five.

Critical Analysis

Where the analogy holds, it holds strongly. The fundamental engineering insight — that trust verification and execution throughput have different optimization targets and should be served by different layers — is not Ethereum-specific. It is a general property of layered system design, well documented in the distributed systems literature [10, 11]. End-to-end arguments in system design, articulated classically by Saltzer, Reed, and Clark, reach the same conclusion: functions placed low in the system should be those that cannot be correctly implemented at the application layer. Trust verification is precisely such a function. Execution throughput is not.

The L1/L2 separation also has a clean analog in the enterprise integration patterns literature. Hohpe and Woolf's pattern language includes the distinction between a system of record and application systems that derive state from it, connected through messaging rather than shared database access [12]. The L2 rollup pattern is, in many ways, this distinction enforced by cryptographic commitments rather than by organizational convention. The architectural move from convention to commitment is meaningful, and it is what gives the L2 pattern its strong correctness properties.

The publication layer is the part of the L1/L2 pattern that most enterprise systems underserve. Most enterprise integrations use a database-as-integration-layer anti-pattern: the core and periphery share a database, the periphery writes directly to tables the core owns, and the verification strategy is implicit trust plus after-the-fact reconciliation. The L2 pattern forces a cleaner interface by making the publication an explicit, signed, append-only artifact. This is the single most valuable steal in the pattern, and it is the steal that requires the least cryptographic machinery to implement.

Counterarguments & Limitations

We surface five limitations that any serious reviewer would raise. First, L2s inherit L1's liveness assumptions. An L2 rollup cannot settle if L1 is unavailable, and an enterprise system that uses the L1/L2 pattern inherits an equivalent dependency: if the core is down, the periphery can keep executing locally but cannot reach settlement. For systems that require hard real-time response — industrial control, vehicle autonomy, life-critical medical — this is a non-trivial constraint, and the design move is to make the periphery capable of safe local operation during core unavailability, not to assume the core will always be available.

Second, the L2 pattern assumes the trust root is genuinely trust-minimized. Ethereum L1 is trust-minimized because its validator set is large, its consensus mechanism is Sybil-resistant through proof-of-stake, and its economic security is observable on-chain. Most enterprise cores are trust-minimized only in the weak sense that they are operated by a single organization that has reputational and legal exposure. The L2 pattern still works in this setting — the publication layer and verification strategy are still useful — but the strong correctness guarantee that L2 rollups inherit from L1 is weaker, because the core is weaker.

Third, the regulatory asymmetry between layers is different. Ethereum's L1 and L2 are both unregulated infrastructure, so the publication layer between them can be designed without regulatory constraint. Enterprise cores and peripheries often sit in different regulatory regimes: a financial settlement core may be subject to banking secrecy, while the periphery (a customer-facing application) may be subject to data residency. The publication layer between them is now a regulatory boundary, not just an architectural one, and its design must respect both.

Fourth, the cost model is different. Ethereum's L2 publication layer is cheap because EIP-4844 blob space is priced separately from execution gas, and because rollups amortize publication cost across many transactions [13]. Enterprise publication layers — append-only audit logs, event streams, evidence stores — do not have an equivalent cost-amortization mechanism unless they are explicitly designed for it. The publication layer can become a cost bottleneck if the periphery commits too frequently or commits too much evidence per transition.

Fifth, the upgrade paths are different. Ethereum upgrades through a coordinated multi-year EIP process; L2 rollups upgrade through their own governance with shorter time horizons. Enterprise cores upgrade through organizational change management that is slower than either. The L1/L2 pattern's velocity asymmetry — slow core, fast periphery — is actually a feature in the enterprise setting, because it forces periphery teams to design for stability around the core's interface contracts. But it is a constraint that needs to be acknowledged in the architecture.

None of these limitations invalidate the steal. They shape how the steal should be implemented. The architectural move is still valid; the cryptographic and economic machinery Ethereum uses to enforce it is what does not transfer cleanly. The honest version of the steal is: take the layering discipline and the publication-layer pattern, leave the proof system and the economic security model, and substitute organizational governance and audit for cryptographic settlement where appropriate. That is a weaker but still useful architectural move, and it is the move we recommend in Series 2 and Series 3.

EDGE AI → LOCAL DECISIONS → BATCH EVIDENCE → CENTRAL VERIFICATION → AUDIT LAYER

Episode 02 — Steal the Rollup Model for Enterprise AI

***Thesis.** The rollup pattern — execute locally, batch evidence, settle periodically, verify cryptographically — is the right architectural answer to the question every enterprise AI deployment is currently fumbling: how to combine local inference speed with central auditability without putting every decision through a centralized control plane.*

Mechanism

A rollup, in the technical sense, is a system that executes transactions off-chain and posts a compressed representation of their effect on-chain. The on-chain component — the L1 contract — holds the canonical state root. The off-chain component — the rollup node — produces batches of transactions, computes the new state root, and publishes both the batch and the new root to L1. Verification takes one of two forms: an optimistic rollup posts the new root and assumes it is correct unless a fraud proof is submitted within a challenge window; a validity rollup posts the new root alongside a cryptographic proof that the transition is valid [14, 15].

The economic insight that makes rollups work is amortization. Verification — whether through fraud proofs or validity proofs — has a fixed cost per batch that does not depend on the number of transactions in the batch. A rollup that batches 10,000 transactions into a single proof amortizes the verification cost across 10,000 payers. The per-transaction cost drops by orders of magnitude compared to executing each transaction on L1 directly. This is the fundamental economic reason rollups exist.

The cryptographic insight is that validity proofs (zk-SNARKs and zk-STARKs) make the verification asymptotically cheaper than the computation they verify [16, 17]. A proof that verifies in milliseconds can attest to the correctness of minutes of computation. The two technologies that made modern rollups possible are, in order: recursive proof composition (which lets multiple proofs be combined into one) and application-specific circuits (which let proof generation be amortized over a domain-specific workload rather than a general virtual machine).

The system-design insight that ties the others together is that the rollup pattern preserves locality. The execution happens where the data is generated. The settlement happens where the trust root lives. The publication layer is the bridge. This is what makes the pattern suitable for geographically distributed, latency-sensitive workloads — the execution is not shipped to the trust root; only the evidence of correct execution is.

Architecture Translation

The enterprise translation is most powerful in the AI governance setting. The dominant pattern in enterprise AI deployments today is a hub-and-spoke model: edge devices (point-of-sale terminals, factory cameras, mobile apps, industrial sensors) collect data and ship it to a central inference service; the central service runs the model and returns a decision; the decision is logged centrally for audit. This pattern has two failure modes that are well known to anyone operating it: the central service becomes a latency bottleneck and a cost bottleneck, and the audit log captures the decision but not the inputs that produced it, which makes post-hoc verification nearly impossible.

Figure 1.2 — Enterprise Rollup Pattern for AI

Edge inference produces local decisions; decisions are batched with cryptographic evidence; the central verification layer settles state and emits audit records.

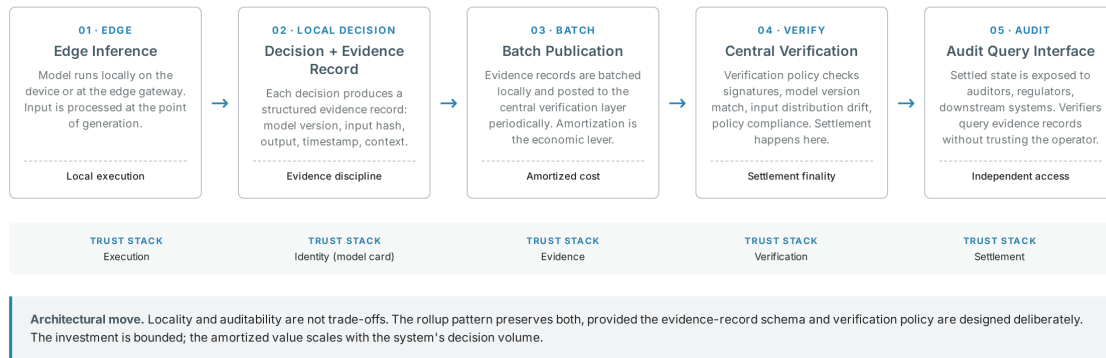


Figure 1.2 — The enterprise rollup pattern for AI. Edge inference produces local decisions; decisions are batched with cryptographic evidence; the central verification layer settles state and emits audit records. Locality is preserved; auditability is preserved; the central plane is not on the inference hot path.

The rollup pattern restructures this. Edge devices run inference locally — the model is shipped to the edge, not the data to the center. The edge device produces a decision, plus a structured evidence record containing the model version, the input hash, the output, the timestamp, and any decision-relevant context. Batches of these records are posted to a central verification layer periodically. The central layer does not re-execute the inference; it verifies the evidence records against the published model card, checks for anomalies, and settles the canonical state of the decision log. The audit layer queries this settled state.

Concretely, the pattern produces five architectural artifacts. The first is the model card, published centrally and versioned. The second is the evidence record schema, which every edge inference produces locally. The third is the batch publication format, which is the wire-level interface between edge and center. The fourth is the verification policy, which specifies what the center checks (signature validity, model version match, input distribution drift, decision-policy compliance). The fifth is the audit query interface, which exposes settled state to reviewers, regulators, and downstream systems.

The pattern's value proposition is that it preserves both the latency property of local execution and the auditability property of central control. Most current enterprise AI architectures trade one for the other. The rollup pattern — stolen faithfully from Ethereum — is the architectural move that lets you keep both, provided you are willing to invest in the evidence record schema and the verification policy. The investment is non-trivial, but it is bounded, and it is amortized across every decision the system makes.

The translation extends naturally to other settings. In industrial IoT, the pattern replaces the typical stream-everything-to-the-cloud model with a batch-evidence-periodically model that reduces bandwidth costs and survives connectivity interruptions. In mobility, the pattern lets vehicle autonomy run locally while preserving fleet-level audit. In healthcare, the pattern lets inference happen at the bedside while preserving regulator-friendly audit trails. In all three, the architectural move is the same: ship the model to the data, ship the evidence to the center, and design the verification layer deliberately.

Critical Analysis

The pattern's strongest feature is its preservation of locality. This is a property most enterprise AI architectures do not preserve, and the cost of losing it is significant: latency-sensitive decisions that round-trip through a central inference service are slow and expensive in a way that compounds with deployment scale. The rollup pattern's edge-execution property is the architectural move that breaks the round-trip cost.

The pattern's second strongest feature is its evidence-record discipline. Most enterprise systems that produce decisions also produce logs, but few produce evidence — structured, signed, verifier-readable records that attest to the inputs, model version, and output of a specific decision. The evidence record is the unit that makes post-hoc verification possible at scale. It is also the unit that makes the regulatory conversation tractable: regulators can audit a sampled set of evidence records rather than the entire decision stream.

The pattern aligns well with the NIST AI Risk Management Framework's emphasis on traceability and the EU AI Act's documentation requirements [18, 19]. It is also compatible with the emerging W3C Verifiable Credentials data model for evidence records, which means the publication layer can be implemented using standardized credential formats rather than bespoke schemas. These alignments are not accidental: the rollup pattern was designed for systems where the verifier is adversarial, and the verification discipline it enforces is stronger than what most regulators currently require.

Counterarguments & Limitations

We surface five limitations. First, rollup exit windows are days, not milliseconds. Ethereum's optimistic rollups have challenge windows of one to two weeks because the economic game theory of fraud proofs requires time for watchers to detect and submit challenges. Enterprise AI decisions that need settlement finality within seconds cannot use the optimistic variant of the pattern; they must use validity proofs, which require cryptographic machinery that is currently expensive to operate at scale [20].

Second, not every enterprise decision needs settlement finality. Many AI decisions are reversible and low-stakes, and the cost of producing evidence records and verification for them is a net loss. The pattern should be applied selectively — to high-stakes, low-reversibility, audit-relevant decisions — not uniformly across all decisions. Most enterprise AI systems make more low-stakes decisions than high-stakes ones, and the pattern's overhead is wasted on the former.

Third, AI determinism is weaker than transactional determinism. A rollup can re-execute a transaction and verify that the resulting state matches the published state root, because transaction execution is deterministic given the input. AI inference is not fully deterministic in the same sense: model outputs can vary across hardware, numerical library versions, and batch composition. Validity proofs for AI inference exist in research form but are not production-ready [21]. The honest version of the pattern, for now, verifies evidence records structurally (does the record conform to the schema? does the model version match the published card?) rather than cryptographically (does the output match what the model would produce on the input?).

Fourth, the cost structure of enterprise AI rollups is inverted from Ethereum's. Ethereum rollups are cheap because L1 publication is cheap relative to L1 execution. Enterprise AI publication layers (audit log storage, evidence record verification) are expensive relative to edge inference. The amortization that

makes Ethereum rollups economic does not yet apply cleanly to enterprise AI, because the unit economics are different. This will change as evidence-record infrastructure matures, but it is the current state.

Fifth, the pattern assumes the edge is capable of producing evidence records of acceptable quality. In practice, edge devices often have constrained compute, intermittent connectivity, and limited tamper resistance. An evidence record produced by a compromised edge device is worse than no evidence record, because it provides false assurance. The pattern needs to be paired with an attestation discipline — hardware roots of trust, secure enclaves, or remote attestation — that is currently immature in most enterprise edge deployments [22].

These limitations do not invalidate the steal. They scope it. The pattern is the right pattern for high-stakes, audit-relevant, latency-sensitive decisions where the edge can be trusted to produce honest evidence. It is the wrong pattern for low-stakes, non-audit-relevant, or fully-centralized decisions. The architecture work is in deciding which decisions fit which category, and that decision is itself the most valuable output of an enterprise rollup design exercise.

FROM L2 TRANSACTION ORDERING TO THE ENTERPRISE DECISION SEQUENCER

Episode 03 — Steal the Sequencer

***Thesis.** Every system with concurrent actors needs an ordering policy. Ethereum's L2 sequencer is the most-studied, most-contested instance of this problem in production. The pattern it suggests — a sequencer layer that controls ordering, priority, authorization, evidence emission, and escalation — is a missing architectural primitive in most enterprise systems.*

Mechanism

An L2 sequencer is the component that determines the order in which user-submitted transactions are included in the rollup's execution. On a technical level, the sequencer receives transactions from users, orders them into blocks, and feeds those blocks to the rollup's execution engine. The ordering decision is significant because transaction order determines state — especially in systems with concurrent financial interactions, where the order in which two transactions execute determines which one succeeds.

The sequencer is the most politically and economically loaded component of the L2 stack. It is the point at which maximal-extractable-value (MEV) is captured or lost, the point at which censorship can be applied or resisted, and the point at which centralization risk concentrates. Most production L2s today run a single-sequencer model: one operator, often the rollup team itself, orders transactions. This is a known centralization point, and the L2 industry's research roadmap is largely organized around removing it [23, 24].

Decentralized sequencer designs — Espresso Systems, Astria, SUAVE, the shared-sequencer pattern — replace the single-sequencer model with a consensus protocol among multiple sequencers. The consensus protocol guarantees liveness (the sequencer set cannot be unilaterally censored) and fair ordering (the sequencer cannot extract MEV by reordering transactions it has already seen). The technical challenge is doing this without adding latency to the ordering path, which would degrade the user experience that centralized sequencing provides.

The force-inclusion escape hatch is the architectural feature that prevents the sequencer from being a single point of censorship. Users whose transactions are not sequenced within a timeout window can submit them directly to L1, where they are eventually included in the rollup's execution regardless of sequencer cooperation. This is a load-bearing design property: it converts sequencer censorship from a hard blockade to a soft delay. The escape hatch is what allows Ethereum L2s to claim censorship resistance under a centralized sequencer — the sequencer can delay, but cannot ultimately block.

Architecture Translation

The steal is to recognize that every enterprise system with concurrent actors has an implicit sequencer, and that the implicit sequencer is usually an accident of implementation rather than a designed primitive. The database's transaction log is a sequencer. The message broker's partition is a sequencer. The work scheduler's priority queue is a sequencer. The API gateway's request router is a sequencer. In each case, ordering is determined, and ordering has consequences, but the ordering policy is rarely treated as a first-class architectural decision.

Figure 1.3 — Enterprise Decision Sequencer

The sequencer is an explicit architectural primitive that controls ordering, priority, authorization, evidence emission, and escalation. Force-inclusion escape hatches live at the boundary.

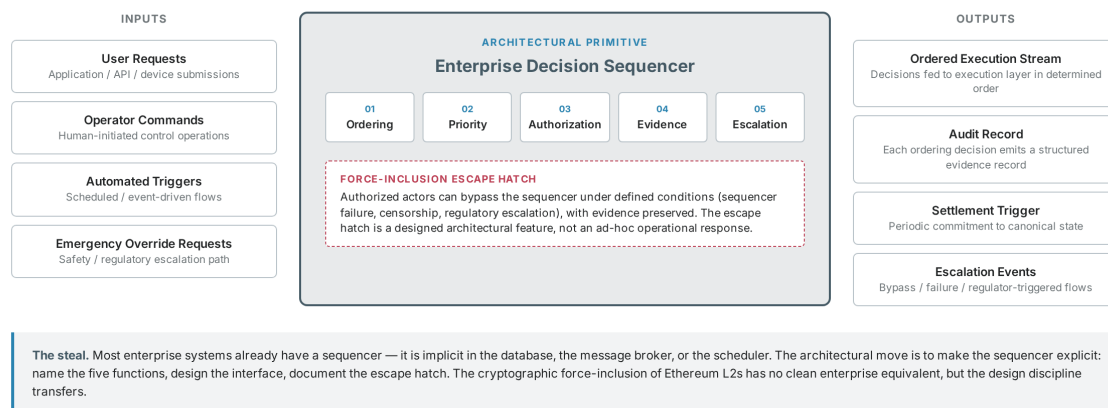


Figure 1.3 — The Enterprise Decision Sequencer. The sequencer is an explicit architectural layer that ingests requests, applies ordering policy, authorizes execution, and emits evidence. Force-inclusion escape hatches live at the boundary, allowing emergency escalation when the sequencer fails or censors.

The Enterprise Decision Sequencer, as a designed primitive, controls five things: ordering (which decision executes first), priority (which decisions preempt others), authorization (which decisions are permitted at all), evidence (what the audit record of each decision contains), and escalation (what happens when the sequencer itself fails). Making these five explicit — rather than implicit in the database or the

scheduler — is the architectural move.

Concrete applications cut across operational settings. In industrial operations, the sequencer controls the ordering of control commands to actuators, the priority of safety overrides, and the authorization of operator interventions. In mobility, the sequencer controls intersection precedence, vehicle routing decisions, and emergency vehicle priority. In financial workflows, the sequencer controls transaction ordering, settlement batching, and dispute resolution. In AI operations, the sequencer controls inference request ordering, model update rollout, and rollback decisions. In each setting, the design question is the same: who decides the order, on what policy, with what evidence, and what happens when the decider fails.

The escape hatch is the part of the steal that most enterprise architectures omit. Ethereum's force-inclusion mechanism is an explicit acknowledgment that the sequencer is a single point of failure and that users must have a fallback path. The enterprise equivalent is an explicit policy for what happens when the central scheduler, ordering service, or workflow engine is unavailable. Most enterprise systems today either lack this policy or implement it as an ad-hoc operational response. The steal is to make the escape hatch a designed architectural feature: a documented path by which authorized actors can bypass the sequencer under defined conditions, with evidence of the bypass preserved.

Critical Analysis

The pattern's most valuable contribution is making ordering policy explicit. In enterprise systems, ordering policy is usually implicit in the database's transaction isolation level, the message broker's delivery semantics, or the scheduler's queue discipline. These choices have real consequences — a read-committed isolation level permits a different class of anomalies than a serializable one — but they are rarely surfaced as architectural decisions. The sequencer pattern forces the surfacing.

The pattern also aligns with the architectural-decision-record (ADR) discipline that mature engineering organizations already practice [25]. An ADR for the sequencer answers: what is the ordering policy, who controls it, what are the priority rules, what is the authorization model, what evidence is emitted, and what is the escape path. These questions are answerable today, in any enterprise, without any cryptographic machinery. The value of the L2 sequencer pattern is that it provides a tested template for what an ADR for ordering should contain.

The MEV question is the part of the L2 pattern that does not translate cleanly. MEV exists in Ethereum because transaction ordering has direct financial value: the order in which two trades execute determines which one captures the arbitrage. Enterprise systems have analogous value at stake in ordering — pricing decisions, resource allocation, queue position — but the value is rarely as cleanly extractable as MEV. The MEV literature is a useful design vocabulary for thinking about ordering value, but the specific economic mechanisms Ethereum uses to redistribute or suppress MEV do not have direct enterprise analogs [26].

Counterarguments & Limitations

We surface five limitations. First, the force-inclusion escape hatch is the most novel part of the L2 sequencer pattern, and it is the part that translates least cleanly. Ethereum's escape hatch works because

L1 is always available and is itself a verifiable trust root. Enterprise systems rarely have a fallback trust root of equivalent strength: if the sequencer is down, the fallback path typically involves a human operator with elevated privileges, which is a weaker guarantee than cryptographic force-inclusion.

Second, MEV has no clean enterprise analog. The economic literature on MEV is rich [27, 28], but the specific redistribution mechanisms — MEV auctions, proposer-builder separation, fair-ordering protocols — are designed for systems where the value of ordering is directly monetizable. Enterprise systems with implicit ordering value (who gets the resource first, whose transaction is processed first) do not have the same redistribution machinery, and importing it is non-trivial because the value is not denominated in a single fungible asset.

Third, single-sequencer failure modes are real and underweighted. A centralized sequencer that fails leaves the system unable to make ordering decisions until it is restored. Ethereum L2s tolerate this because the escape hatch exists and because the time-to-recovery is short in the relevant operational sense. Enterprise systems with a single sequencer and a weak escape hatch inherit the same failure mode, often without acknowledging it. The architectural decision here is between accepting the single-sequencer risk (with documented mitigation), investing in multi-sequencer consensus (with the complexity that implies), or designing the periphery to operate without the sequencer during outages.

Fourth, the pattern assumes that ordering is, in fact, a system-wide property. In many enterprise systems, ordering is local: each subsystem orders its own work, and the cross-subsystem ordering question is moot because the subsystems are loosely coupled. The sequencer pattern applies to systems where there is a meaningful global ordering question — financial settlement, real-time control, multi-party coordination — and not to systems where local ordering suffices. Applying the pattern where it does not apply adds architecture without adding value.

Fifth, the political economy of the sequencer — who controls ordering, who benefits from it, who is harmed by the choices made — is a real architectural consideration that the L2 pattern surfaces but does not resolve. In Ethereum L2s, the sequencer is often the rollup team itself, which has commercial incentives that shape the ordering policy. In enterprise systems, the sequencer-equivalent is often the platform team, the operations team, or a vendor. The pattern makes the political economy explicit; it does not make the politics go away.

These limitations suggest a tempered version of the steal. Take the explicit ordering-policy discipline. Take the evidence-emission requirement. Take the escape-hatch design move, even if implemented through organizational rather than cryptographic means. Leave the MEV-redistribution machinery. Leave the cryptographic force-inclusion. Acknowledge the political economy. The result is a sequencer pattern that is meaningfully stronger than what most enterprise systems have today, without pretending to be Ethereum.

FROM L2 PUBLICATION LAYERS TO THE ENTERPRISE EVIDENCE AVAILABILITY LAYER

Episode 04 — Steal Data Availability

***Thesis.** “I have a result” is not enough. The verifier must be able to independently access the information needed to check the result. Ethereum's data availability problem is the most precise formulation of this requirement in the distributed systems literature, and the Enterprise Evidence Availability Layer it suggests is a missing primitive in AI governance, regulatory compliance, and audit.*

Mechanism

Data availability is the problem of proving that the data needed to verify a state transition is, in fact, available to anyone who wants to verify it. It is distinct from data publication (the act of posting the data) and data storage (the act of keeping the data). The distinction matters: a system can publish data without making it durably available, and a system can store data without making it available to verifiers. Data availability is the verifier-side property that the data needed for verification can actually be obtained.

In Ethereum's L2 architecture, the data availability problem arises because rollups publish state transitions to L1, but the inputs needed to verify those transitions — the actual transactions — must be published separately. If a rollup publishes only the new state root and not the transaction data, verifiers cannot check that the transition is correct. The rollup could claim any state it wanted. Data availability sampling, danksharding, and the blob-publication mechanism in EIP-4844 exist to solve this problem: they guarantee that the data needed for verification is published in a form verifiers can reconstruct [29, 30].

The dedicated data availability layer — Celestia, EigenDA, Avail, and the equivalent Ethereum-native mechanisms — abstracts this property into a separable service [31, 32]. A rollup publishes its data to the DA layer, the DA layer guarantees availability through cryptographic sampling, and the rollup's L1 contract references the DA layer's commitment. This separation lets the DA layer be optimized for cheap, high-throughput publication while the L1 trust root remains focused on settlement.

The cryptographic insight that makes modern DA layers practical is data availability sampling: a verifier can check that a large dataset is available by sampling a small number of random chunks, without downloading the entire dataset. If any sampled chunk is missing, the verifier can prove the data is unavailable. If all sampled chunks are present, the data is available with high probability. This is what makes DA layers cheap to verify — the cost of verification does not scale linearly with the size of the data being verified.

Architecture Translation

The enterprise translation is to recognize that every audit-relevant decision produces a data availability problem, and that most enterprise systems solve it incorrectly by default. The typical enterprise pattern is:

the decision system writes a log entry to a database, the log entry contains the decision and a timestamp, and the audit team queries the database when they need to verify. This pattern fails three of the data availability requirements that Ethereum's DA layer is designed to meet.

Figure 1.4 — Enterprise Evidence Availability Layer

Every audit-relevant decision produces an evidence record. The availability layer guarantees verifiers can reconstruct the inputs needed to check the decision.

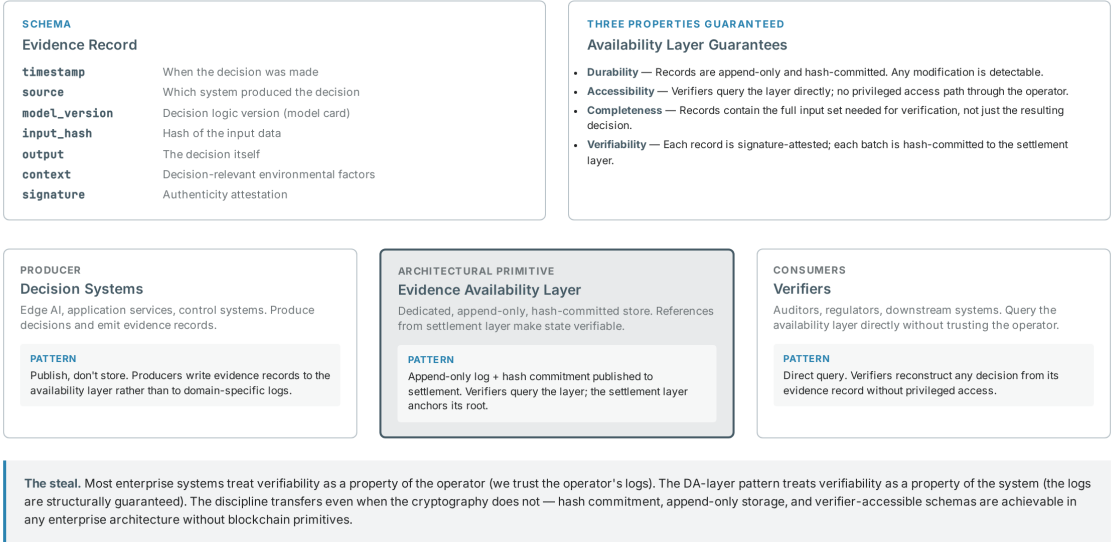


Figure 1.4 — The Enterprise Evidence Availability Layer. Every audit-relevant decision produces an evidence record. The evidence is published to a dedicated availability layer that guarantees verifiers can reconstruct the inputs needed to check the decision. The settlement layer references the availability layer's commitment.

The first failure is durability: log entries in a database can be modified or deleted by anyone with database access. An audit team that queries the database two years later cannot verify that the record they retrieve matches the record that was originally written. The L2 DA layer solves this with cryptographic commitments: the published data is bound to a hash that the L1 contract references, and any modification is detectable.

The second failure is accessibility: log entries are often stored in a database that requires privileged access to read. Verifiers (auditors, regulators, downstream systems) must either be granted database access or rely on the system's operator to provide extracts. The L2 DA layer solves this by making published data available to anyone who can read the L1 contract — there is no privileged access path.

The third failure is completeness: log entries typically record the decision and a few context fields, but not the full input set needed to verify the decision was correct. An audit team that retrieves a log entry cannot reconstruct what the system was actually deciding. The L2 DA layer solves this by requiring that the full transaction data be published — not just the resulting state.

The Enterprise Evidence Availability Layer, as an architectural primitive, addresses all three failures. It is a dedicated, append-only, hash-committed store for evidence records produced by audit-relevant decisions. Every decision in the system that has audit relevance produces an evidence record conforming to a published schema, and the record is written to the availability layer rather than to a domain-specific log. The settlement layer references the availability layer's commitment, so the canonical state of the

decision log is verifiable without querying the availability layer directly.

Concretely, an evidence record contains: a timestamp, a source identifier (which system produced the decision), a model or policy version (which decision logic was applied), the input (the data the decision was based on, hashed if the input is large), the output (the decision itself), the decision context (any additional factors that shaped the decision), and a signature attesting to the record's authenticity. The availability layer guarantees these records are durable, accessible, and complete for any verifier who needs to audit a decision.

Critical Analysis

The pattern's strongest feature is that it makes verifiability a property of the system rather than a property of the operator. In a typical enterprise system, verifiability depends on the operator's willingness and ability to provide complete, unmodified records to auditors. In a system with an evidence availability layer, verifiability is structurally guaranteed: the records are committed to a hash that is referenceable from the settlement layer, and any modification is detectable.

The pattern aligns well with the existing audit and compliance literature. ISO 27001's requirements for log integrity, the NIST AI Risk Management Framework's traceability requirements, and the SEC's electronic record-keeping rules all impose some version of the durability, accessibility, and completeness properties the DA layer is designed to provide [33, 34]. The DA pattern offers a single architectural primitive that satisfies all three, rather than the patchwork of log-management tooling that most enterprises currently use to meet these requirements.

The pattern also fits cleanly with the W3C Verifiable Credentials data model, which provides a standardized format for signed, attestable claims [35]. Evidence records in the availability layer can be expressed as verifiable credentials, which gives the pattern interoperability with the broader identity and attestation ecosystem without locking it into a particular vendor's format.

Counterarguments & Limitations

We surface five limitations. First, Ethereum's data availability layer assumes an adversarial model where the operator of the publishing system has a rational incentive to withhold data. Enterprise systems rarely have the same adversarial structure: the operator and the verifier are usually part of the same organization, and the operator's incentive to withhold data is shaped by internal politics rather than economic game theory. The DA layer's cryptographic machinery is stronger than what the enterprise threat model requires, which means it is also more expensive than necessary.

Second, enterprise data is governed by privacy law rather than game theory. GDPR's data subject access requests, CCPA's deletion rights, and the emerging AI Act's documentation requirements impose obligations on data retention and modification that conflict with the append-only, hash-committed property of a DA layer. A DA layer that supports evidence records containing personal data must be designed to comply with these obligations, which means it cannot be purely append-only in the cryptographic sense.

Third, storage cost tradeoffs are different. Ethereum's DA layer is cheap because blob space is a bounded resource that is priced by market mechanisms. Enterprise evidence stores are expensive relative to the

volume of decisions they would need to record, and the cost of long-term retention (10+ years for some regulatory regimes) compounds. The amortization that makes Ethereum DA layers economic does not apply cleanly to enterprise evidence stores, where the cost is per-record rather than per-batch.

Fourth, the pattern assumes that evidence records are complete in the sense that they capture everything needed for verification. In practice, the decision context that shapes an enterprise AI decision is often implicit — operator overrides, system state, environmental conditions — and capturing it requires explicit engineering investment that most systems do not make. A DA layer that stores incomplete evidence records is worse than no DA layer, because it provides false assurance.

Fifth, the pattern's verification model assumes that verifiers can re-execute decisions. For AI decisions, this is non-trivial: re-execution requires the same model version, the same input, and often the same execution environment. Most enterprise AI deployments do not preserve all three, which means the DA layer's evidence records can be verified structurally (the record is intact and signed) but not semantically (the decision was correct). The honest version of the pattern for AI is a structural-verification layer with explicit acknowledgment that semantic verification is currently out of reach for most deployments.

The pattern is still valuable, with these limitations acknowledged. The architectural move from “log entries in a database” to “evidence records in a dedicated availability layer with hash commitments” is meaningful, even if the cryptographic machinery is weaker than Ethereum's. The value is in the discipline, not the cryptography.

FROM L1 FINALITY TO THE TRUST STACK AND THE SETTLEMENT LAYER AS GOVERNANCE PRIMITIVE

Episode 05 — Steal Settlement

***Thesis.** Not every decision needs central approval. But important decisions need a trusted final settlement layer that other systems can reference as authoritative. Ethereum's L1-as-settlement pattern is the canonical formulation of this idea, and the Trust Stack it suggests — Identity → Execution → Evidence → Verification → Settlement — is the framework we recommend for thinking about where trust should live in any modern digital system.*

Mechanism

Settlement, in Ethereum's layered architecture, is the property that a state transition is final — it cannot be reversed except through another state transition that the system accepts. L1 provides settlement to L2 rollups: once a rollup's state root is finalized on L1, the rollup cannot revert that state without reverting L1, which requires social consensus at the protocol level and is essentially never done. Settlement is the strongest guarantee in the stack.

The settlement property depends on L1's consensus and economic security. Ethereum L1 finality is provided by Gasper consensus, which gives 2-epoch finality (about 13 minutes) under normal conditions [36]. Once final, a state transition can be reverted only by an attack that controls more than one-third of the validator set's stake — an attack that would destroy billions of dollars of value and is therefore considered economically infeasible. The strength of L1 settlement is the strength of this guarantee.

L2 rollups inherit settlement finality from L1 in a specific sense. Once an L2 state root is posted to L1 and the challenge window (for optimistic rollups) has expired or the validity proof (for validity rollups) has been verified, the L2 state is considered final. The L2 cannot revert that state without reverting L1. This is what makes L2 state useful as a settlement layer for downstream applications: a payment made on an L2 is, after settlement, as final as a payment made on L1.

Bridging between settlement layers is where the model gets fragile. A bridge between two L2s — say, Optimism and Arbitrum — must trust both L2s' settlement guarantees, plus the bridge protocol itself. Bridges are the most-attacked component in the Ethereum ecosystem precisely because they concentrate trust: a single bug in a bridge can allow an attacker to mint assets on one side without burning them on the other, compromising the settlement guarantees of both chains the bridge connects [37]. The settlement layer is only as strong as its weakest bridge.

Architecture Translation

The enterprise translation is the Trust Stack: a five-layer model that locates trust functions in a deliberate hierarchy. At the bottom, Identity — who or what is making a decision. Above it, Execution — the actual decision logic. Above that, Evidence — the records that prove what was decided and why. Above that, Verification — the process by which evidence is checked. At the top, Settlement — the canonical state that other systems reference as authoritative.

Figure 1.5 — The Trust Stack

Trust is the apex property; settlement is where trust is recorded. The stack answers: where should trust live in a modern digital system?

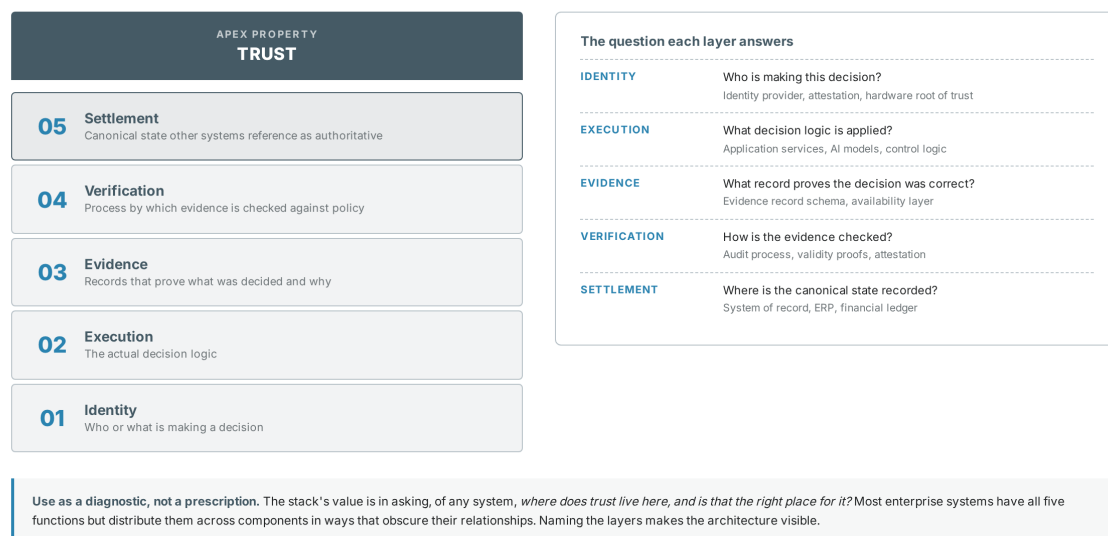


Figure 1.5 — The Trust Stack. Trust is the apex property; settlement is the layer where trust is recorded. Identity, execution, evidence, and verification are the supporting layers. The stack answers the question: where should trust live in a modern

digital system?

The architectural question the stack asks is: at which layer should each trust function live? Identity, in most enterprise systems, lives in an identity provider that authenticates users and services. Execution lives in application services. Evidence, in well-designed systems, lives in an availability layer (as discussed in Episode 04). Verification lives in audit and compliance processes. Settlement — the canonical state that other systems reference — lives in the system of record, typically an ERP, a financial ledger, or a regulatory filing.

The steal is in two parts. First, make each layer explicit and named. Most enterprise systems have all five functions, but the functions are distributed across components in ways that obscure their relationships. Naming the layers makes the architecture visible. Second, recognize that settlement — the top layer — should be small, conservative, and slow. The temptation in enterprise architecture is to make the system of record also the execution system, so that every decision flows through the canonical store. This is the same anti-pattern as putting all execution on L1 in Ethereum: it makes the trust root expensive and slow, and it makes the periphery unable to operate when the trust root is unavailable.

The Trust Stack pattern is the integrating framework for the other steals in this series. The layering model of Episode 01 is the Core/Periphery split between Settlement and Execution. The rollup pattern of Episode 02 is the Execution-Evidence-Verification loop. The sequencer pattern of Episode 03 is the ordering policy for Execution. The data availability pattern of Episode 04 is the Evidence layer's availability guarantee. Each pattern is a specialization of one layer of the stack. The stack itself is the framework that locates the patterns in a single coherent architecture.

Critical Analysis

The Trust Stack's strongest claim is that it makes trust a first-class architectural concern rather than an emergent property. In most enterprise systems, trust is implicit: the system of record is trusted because it is the system of record, the identity provider is trusted because it is the identity provider, and the audit process is trusted because it is the audit process. The stack makes the trust relationships explicit and locatable, which makes them auditable.

The pattern aligns with the NIST Zero Trust Architecture specification [38], which similarly treats trust as a first-class concern that must be explicit and verifiable. The Trust Stack and Zero Trust differ in emphasis: Zero Trust focuses on the identity and access layers, while the Trust Stack focuses on the settlement and verification layers. They are complementary — a complete enterprise architecture needs both.

The stack also makes a useful distinction between trust and trustworthiness. Trust is the property the system grants to a component (we accept this component's outputs as authoritative). Trustworthiness is the property the component actually has (this component's outputs are, in fact, correct). Settlement is where trust is recorded. Verification is where trustworthiness is checked. The two are not the same, and the stack makes their relationship visible.

Counterarguments & Limitations

We surface five limitations. First, Ethereum settlement is final because of economic stake: validators have billions of dollars of ETH at risk, and reverting finality would require destroying that value. Enterprise settlement is rarely backed by equivalent economic security. A financial ledger's finality is backed by regulatory and reputational consequences, which are real but weaker than cryptographic economic stake. The Trust Stack pattern does not give enterprise settlement the strength of L1 finality; it just makes explicit where the weaker settlement guarantee lives.

Second, enterprise settlement is often reversible in ways L1 settlement is not. A financial ledger can be corrected through a reversing entry. A regulatory filing can be amended. A system-of-record update can be rolled back through a database migration. The Trust Stack pattern does not change this — it makes the settlement layer explicit, but the settlement layer's guarantee is still weaker than L1's.

Third, settlement cost is rarely zero in enterprise systems. Ethereum L1 settlement costs gas per transaction, but the gas market makes this cost observable and explicit. Enterprise settlement costs are typically hidden in operational overhead: the reconciliation team, the audit team, the compliance team. The Trust Stack pattern makes these costs visible, but it does not eliminate them.

Fourth, the stack assumes that the settlement layer is a single, identifiable component. In practice, enterprise systems often have multiple settlement layers — the financial ledger for monetary state, the inventory system for physical state, the contract system for legal state — and these layers do not always agree. Cross-system reconciliation is a non-trivial problem that the Trust Stack pattern does not solve; it just makes the disagreement visible.

Fifth, the bridging problem that plagues Ethereum L2s also plagues enterprise settlement layers. When two systems each maintain their own settlement layer and need to coordinate (a payment in the financial ledger and a shipment in the inventory system, say), the coordination is a bridge, and bridges are the weakest link. Enterprise integration patterns provide vocabulary for these bridges [39], but they do not solve the settlement-fragmentation problem. The Trust Stack pattern acknowledges the problem; it does not resolve it.

The honest version of the steal is this: take the stack as a thinking framework, not as a complete architecture. Use it to locate trust functions in your existing systems. Use it to identify where settlement is implicit, where verification is underweight, and where evidence is incomplete. The stack is most valuable as a diagnostic — a way to ask, of any system, “where does trust live here, and is that the right place for it?” It is less valuable as a prescription, because the right place for trust in any given system depends on the system's specifics, and the stack cannot answer that question for you. The stack makes the question askable; it does not make the answer easy.

WHERE MONEY ENTERS THE CONVERSATION. NOT “IS X TECHNICALLY BETTER?” BUT “WHEN DOES X ACTUALLY MAKE ECONOMIC SENSE?”

Series 2 — Architecture ROI

Thesis. *Every architecture decision is a capital allocation. This series transfers the analytical framework to make that allocation explicit, defensible, and tied to outcomes a CFO will recognize.*

Series 1 transferred the architectural patterns. Series 2 transfers the economic reasoning that determines whether those patterns are worth deploying. The two are paired by design: an architecture pattern without an economic model is a hobby project, and an economic model without an architecture pattern is a spreadsheet. We do not want our readers building either.

The series is structured around five questions we believe every technical leader should be able to answer about any proposed architecture: when is an L2 actually cheaper, who captures the transaction economics, when does the proof justify its cost, what is the true cost of verification, and what is the build-versus-buy-versus-rollup decision framework. Each episode gives the analytical tools to answer one of these questions, with the data and references to support a defensible answer.

Blueprint depth means each episode is structured: thesis, five key mechanisms, critical caveats, counterarguments, references. This is not the long-form prose of Series 1. It is a working document, designed to be consulted during an actual architecture review rather than read end to end.

THE COST DECOMPOSITION EVERY TECHNICAL LEADER SHOULD RUN BEFORE COMMITTING

Episode 01 — When Is an L2 Actually Cheaper?

Thesis. *“L2s are cheaper than L1” is the most-quoted and least-examined claim in the rollup ecosystem. The honest answer is that L2s are cheaper in some regimes and more expensive in others, and the regime question is the one that determines whether the architecture pays off.*

Thesis

An L2 is cheaper than L1 only when the cost decomposition works out favorably, and the decomposition is more complex than the marketing claims suggest. The total cost of operating an L2-anchored workload includes execution compute, data publication, settlement, proof generation, infrastructure, and operations. Each component has a different cost profile, and the combination determines whether the L2 is, in fact, cheaper than the L1 alternative. We provide the decomposition, the data sources, and the

methodology to compute it.

Five Key Mechanisms

1. The cost decomposition formula. The total cost of an L2-anchored workload is: $\text{Total Cost} = \text{Execution} + \text{Data} + \text{Settlement} + \text{Proof} + \text{Infrastructure} + \text{Operations}$. Execution is the cost of running the rollup node. Data is the cost of publishing state to L1 (blob gas, post-EIP-4844 [1]). Settlement is the L1 gas paid for state-root publication. Proof is the cost of generating fraud or validity proofs. Infrastructure is the cost of sequencers, validators, monitoring, and redundant capacity. Operations is the human cost of running the system.

2. Per-L2 comparison data. L2Beat publishes fee, throughput, and risk data for major L2s [2]. As of mid-2024, median transaction fees range from sub-cent (zkSync Era during low-load periods) to several cents (Arbitrum One during congestion). The variation is one to two orders of magnitude, which is large enough to swing an architecture decision. Cheapest is not always right — risk, liveness, and ecosystem fit also matter — but cheapest is the floor.

3. Floor cost dynamics. L2 fees have a floor set by fixed costs (sequencer operation, L1 publication amortized across transactions). Below the floor, the L2 operator is subsidizing. Above the floor, the L2 is profitable. Sustainable low fees require either high throughput (to amortize fixed costs) or subsidy (which is temporary). The floor is the relevant number for long-term cost modeling, not the headline fee.

4. Batching amortization. The fundamental economic lever in L2 design is batching: more transactions per batch means more payers sharing fixed costs. Optimism's and Arbitrum's per-transaction fees drop by an order of magnitude when batch sizes are large [3]. For enterprise systems with steady-state workloads, batching economics favor L2 architectures over L1 architectures by a similar margin. For workloads with low transaction rates, the batching advantage shrinks and the L2 fixed cost can exceed the L1 variable cost.

5. L1 gas price sensitivity. L2 fees are not independent of L1 gas prices, because L2 publication cost is paid in L1 gas. When L1 gas spikes (as it did during NFT drops, DeFi summer, and various meme-coin events), L2 fees spike too, by a smaller but still significant factor. Enterprise cost models that assume L2 fees are stable under all L1 conditions are wrong. The correct model is to assume L2 fees track L1 gas with a dampening factor of 5-20x, depending on the L2's batch efficiency.

Critical Caveats

Price is not cost. The fee a user pays is a price set by the L2 operator, often subsidized. The cost of running the L2 is the operator's cost, which is opaque. Enterprise buyers should not assume that the prices they see today will persist; many L2 operators are running at a loss to acquire market share. When subsidy ends, prices will rise to cover costs, and the architecture that looked cheap under subsidy may not look cheap without it [4].

Off-peak fees distort. L2 fees are typically measured during low-congestion periods, which makes L2s look cheaper than they are during real workloads. Enterprise workloads often have peak periods (month-end, market open, campaign launches) where fees are higher. Cost models should use peak-period fees, not median fees, for capacity planning.

Proof amortization depends on volume. Validity proofs are expensive to generate but cheap to verify. The amortization works only when the proof is shared across many transactions. For low-volume workloads, validity proofs are not economic; optimistic verification (cheaper to generate, expensive only on challenge) is the right choice. The crossover point is workload-specific.

Counterarguments & Limitations

The cheapest L2 is not always the right L2. L2Beat's risk framework [2] documents that low fees often correlate with centralization, weak security councils, or unproven fault-proof systems. The cost-optimal choice for an enterprise workload may be a slightly more expensive L2 with stronger risk properties. The cost decomposition we provide does not capture risk-adjusted cost, which is the metric a CFO should actually use.

Cost models break under congestion. The cost decomposition we describe is a steady-state model. Under congestion — which is exactly when cost matters most — the model's assumptions fail. Fees spike non-linearly, proof generation queues, and sequencer throughput becomes the binding constraint. Enterprise cost models that use steady-state fees for capacity planning will understate peak-period cost by 5-50x.

Total cost of ownership must include operational overhead. The decomposition we provide focuses on direct per-transaction costs. TCO also includes the salaries of the team operating the L2 integration, the monitoring infrastructure, the incident response capacity, and the ongoing integration maintenance. For most enterprise deployments, operational overhead is 2-5x the direct per-transaction cost. Architecture decisions made on direct-cost basis alone will understate true cost by this factor.

Finally, the cost decomposition assumes the L2 is operating as designed. Real-world L2 incidents — sequencer outages, bridge hacks, proof-generation bugs — impose costs that the steady-state model does not capture. A defensible architecture decision includes an incident-cost estimate based on historical L2 incident frequency and severity. We do not have space in this blueprint to develop that estimate fully, but it is necessary for any board-level cost decision.

WHOEVER CONTROLS ORDERING CAPTURES THE VALUE. THE ENTERPRISE QUESTION IS WHO THAT SHOULD BE.

Episode 02 — The Sequencer Economics

***Thesis.** Sequencer economics is the question of who captures the economic surplus generated by ordering decisions. In Ethereum L2s, this is the MEV question, and it is one of the most studied problems in the field. In enterprise systems, the equivalent question — who controls priority, who captures the value of priority, who pays for the consequences of ordering choices — is rarely asked. It should be.*

Thesis

Every system with concurrent transactions has a sequencer. The sequencer's ordering choices create or destroy value. In Ethereum L2s, this value is partially captured by the sequencer operator as MEV. In enterprise systems, the value is captured by whoever controls the equivalent ordering function — usually implicitly, and usually without an economic model. We argue that this is a missed opportunity and a hidden risk.

Five Key Mechanisms

1. Transaction fees. The most visible sequencer revenue stream. L2 sequencers charge users a per-transaction fee, set by the L2 operator. Fee revenue scales with transaction volume and is the primary revenue source for most L2 teams. For enterprise analogs, the equivalent is the per-request fee charged by API gateways, message brokers, or workflow engines. The fee structure (flat, tiered, dynamic) is an architectural decision that shapes who uses the system and how.
2. MEV. Maximal extractable value is the value a sequencer can extract by reordering, inserting, or deleting transactions it has seen. In Ethereum L2s, MEV is substantial: research estimates suggest it accounts for 10-30% of sequencer revenue on major L2s [5]. The academic literature on MEV is rich, with foundational work by Daian et al (Flash Boys 2.0) and subsequent contributions on MEV auction design and proposer-builder separation [6, 7].
3. Infrastructure cost. The cost of operating a sequencer is substantial: high-availability compute, low-latency networking, redundant capacity for failover. Espresso Systems and Astria have published cost models for shared sequencer infrastructure [8, 9]. The infrastructure cost is fixed, not per-transaction, which is why batching is the dominant economic lever.
4. Operational cost. Beyond infrastructure, sequencers require 24/7 operations teams, monitoring, incident response, and ongoing protocol upgrades. This is the human cost of running a critical-path component. For enterprise systems, the equivalent is the cost of the team that operates the central scheduler, message broker, or workflow engine. This cost is typically underweighted in architecture decisions.
5. Centralization rent. A centralized sequencer can extract rent by setting fees above cost, by capturing MEV without redistribution, and by leveraging its position to favor certain transactions. This rent is the economic incentive for the L2 industry to move toward decentralized sequencing — not because decentralization is technically superior, but because it redistributes rent [10]. The enterprise analog is the rent extracted by platform teams that control critical infrastructure — an underexamined economic dynamic in most enterprises.

Critical Caveats

Decentralized sequencing redistributes but adds overhead. Consensus protocols for multi-sequencer systems add latency and complexity. The trade-off is between rent extraction by a centralized operator and operational overhead of a decentralized operator. The right answer depends on the workload's tolerance for each.

MEV auction design matters. The specific mechanism by which MEV is captured and redistributed — MEV-Boost, order-flow auctions, encrypted mempools — has different distributional consequences. MEV-Boost, Ethereum's current de facto standard, gives builders significant power. Alternative designs redistribute to users or to the protocol itself. The design choice is value-laden, not just technical.

Enterprise ordering value is real but rarely monetized. In a financial settlement system, transaction ordering determines which trades execute at which prices — a direct analog to MEV. In a SaaS API, request ordering determines which users get capacity during congestion — a service-quality analog. In both cases, the ordering decision creates value, but the value is typically captured by the operator rather than redistributed to users. The architecture question is whether this capture is intentional and disclosed.

Counterarguments & Limitations

Enterprise analogs lack an open market for ordering rights. Ethereum's MEV market is open: anyone can become a builder, anyone can bid for order flow, anyone can extract MEV subject to protocol rules. Enterprise systems do not have equivalent open markets for ordering, which means the redistribution mechanisms that Ethereum has developed do not translate. The pattern's economic machinery depends on a market structure that enterprises typically do not have.

Sequencer economics is a young field. The literature on MEV is five years old at most. Cost models are preliminary, and the field has not yet developed the mature analytical tools of older areas of distributed systems economics. Enterprise decisions based on this literature should treat the numbers as orders of magnitude, not as precise estimates.

MEV measurement is contested. Different researchers use different methodologies to estimate MEV volume, and the estimates vary by 2-3x. Enterprise cost models that use a single MEV estimate should acknowledge the range and the uncertainty.

The pattern assumes the sequencer is, in fact, the binding ordering constraint. In many enterprise systems, the binding constraint is upstream (the database, the API gateway, the application framework) rather than at a designed sequencer layer. Sequencer economics applied to a system where the sequencer is not the binding constraint produces analysis without leverage.

Finally, the political economy of who captures ordering value is a question that the L2 literature surfaces but does not resolve. In Ethereum L2s, the answer is “the sequencer operator, with some redistribution to the protocol.” In enterprises, the answer is typically “the platform team, with no redistribution.” Whether this is the right answer is a question for the organization's leadership, not for the architecture team. The architecture team's job is to make the question askable.

THE INEQUALITY THAT DETERMINES WHETHER ZK ADDS VALUE: PROOF COST < VALUE OF REDUCED TRUST

Episode 03 — ZK: When the Proof Is Worth Paying For

***Thesis.** Zero-knowledge proofs are not always worth their cost. They are worth their cost when the value of reduced trust exceeds the cost of generating the proof. This is a workload-specific and stake-specific decision, and the field's enthusiasm has consistently outrun its economic analysis.*

Thesis

The decision to use zero-knowledge proofs in an architecture is governed by an inequality: proof cost < value of reduced trust/risk. When the inequality holds, ZK is the right choice. When it does not hold, ZK is expensive cryptography applied to a problem that did not need it. We provide the framework to evaluate the inequality for a given workload, with the data sources and decision criteria to make the evaluation defensible.

Five Key Mechanisms

1. Prover economics. The cost of generating a zk-SNARK or zk-STARK proof has dropped by orders of magnitude over the past five years, but it remains non-trivial. A Plonk proof for a typical rollup batch takes minutes to generate on specialized hardware [11]. Halo2 proofs for application-specific circuits are faster but require circuit engineering. The cost is fixed per proof, regardless of how many transactions the proof attests to, which is why amortization through batching is the key economic lever.
2. Recursion. Recursive proof composition — proofs that verify other proofs — allows the cost of proof generation to be amortized across many proofs, each attesting to a small piece of work [12]. Nova and SuperNova are the current state of the art. Recursion makes ZK economic for high-throughput workloads where individual proofs would be too expensive, but it adds complexity to the proof system and requires specialized engineering.
3. Application-specific circuits vs general-purpose VMs. Proofs for application-specific circuits (Halo2, Circom, Noir) are cheaper to generate than proofs for general-purpose VM execution (RISC Zero, SP1, zkEVM). The trade-off is engineering effort: application-specific circuits must be hand-built for each workload, while general-purpose VMs accept arbitrary code [13, 14]. For enterprise deployments with stable, well-defined workloads, application-specific circuits are typically economic. For varied workloads, general-purpose VMs are the right choice despite their higher per-proof cost.
4. Risk-adjusted value of verifiability. The value of reduced trust is the risk-adjusted cost of the bad outcomes that verification prevents. For a low-stakes workload, the risk-adjusted value is low, and ZK is not worth its cost. For a high-stakes workload (settlement, regulatory audit, life-critical decisions), the

risk-adjusted value is high, and ZK may be economic even at current proof costs. The decision is workload-specific and stake-specific.

5. Network effects in proof systems. The cost of ZK infrastructure is falling because of network effects: shared proving infrastructure, recursive composition, and amortization across multiple users all reduce per-user cost. Enterprises that adopt ZK early benefit from the network effects' continuation; enterprises that wait benefit from the cost reduction that has already happened. The decision to wait is defensible; the decision to adopt early is defensible; the decision to ignore ZK entirely is increasingly hard to defend.

Critical Caveats

Proof generation is amortizable, but circuit development cost is fixed. Building a ZK circuit for an application requires specialized engineering that is expensive and scarce. A typical application-specific circuit takes 3-6 months to develop and audit. For workloads with short lifetimes, the fixed cost dominates; for workloads with long lifetimes, the amortized cost is acceptable.

Trust reduction value is context-dependent. The same ZK proof may be worth a great deal in a regulated context (where the alternative is human audit) and worth very little in a trusted-internal context (where the alternative is a database query). The inequality must be evaluated for each deployment context, not in the abstract.

Proof cost is currently opaque. Published proof costs are typically benchmark costs, not production costs. Production proof generation involves infrastructure costs, operational costs, and reliability investments that benchmarks do not capture. Enterprise cost models should multiply benchmark costs by 2-5x to estimate production costs realistically.

Counterarguments & Limitations

ZK is not always worth it. For low-stakes or already-trusted flows, the inequality does not hold, and ZK is expensive cryptography applied to a problem that did not need it. The field's enthusiasm for ZK — driven by venture investment, conference circuit dynamics, and genuine technical progress — has consistently overweighted the cases where ZK is economic and underweighted the cases where it is not.

Proof cost is currently opaque. There is no widely accepted methodology for benchmarking proof costs across systems, and the costs that are published are often best-case figures from the system's authors. Independent benchmarks (like aRIC, ZK Bench) are emerging but not yet authoritative. Enterprise decisions should treat published proof costs as lower bounds, not as expected costs.

Circuit security is not yet a solved problem. ZK circuits have bugs. Soundness bugs (where a malicious prover can convince a verifier of a false statement) have been found in production circuits. The audit infrastructure for ZK circuits is immature relative to the audit infrastructure for traditional code. Enterprises adopting ZK should plan for circuit bugs and should budget for ongoing audit costs.

Regulatory acceptance is not automatic. ZK proofs are a new cryptographic primitive, and regulators have not yet developed consistent guidance on their use. Some regulators (the EU AI Act's draft text, certain SEC no-action letters) have begun to acknowledge cryptographic attestation, but the regulatory framework is still emerging. Enterprises that adopt ZK for compliance purposes should expect regulatory

engagement to be a multi-year process.

Finally, ZK is not the only way to reduce trust. Multi-party computation, trusted execution environments, and traditional audit each reduce trust through different mechanisms and at different costs. The architecture question is not “should we use ZK” but “what is the cheapest trust-reduction mechanism that meets our requirements.” ZK is one option among several, not the default.

TRUST AND VERIFICATION AS ECONOMIC SUBSTITUTES AT THE MARGIN

Episode 04 — The Cost of Verification

***Thesis.** Trust is not free, and verification is not free. The two are substitutes at the margin: more trust reduces the need for verification, and more verification reduces the need for trust. The architecture decision is to choose the mix that minimizes total cost for a given level of assurance.*

Thesis

Every system that makes decisions makes them under some combination of trust (we accept the decision because of who made it) and verification (we accept the decision because we have checked it). The two are economic substitutes at the margin, and the architecture question is which mix to use for which decisions. The cost decomposition we provide lets a technical leader make this trade explicit rather than implicit.

Five Key Mechanisms

1. Trust cost = expected loss $\times$ probability of betrayal. Trust is not free; it has an expected cost equal to the loss that would occur if the trusted party acted against the trust, multiplied by the probability of that betrayal. For a high-stakes decision trusted to a low-probability-of-betrayal party, trust is cheap. For a high-stakes decision trusted to a high-probability party, trust is expensive. The cost is often hidden, realized only when betrayal occurs.
2. Verification cost = proof + verification compute. Verification is also not free; it has a cost equal to the cost of producing the proof plus the cost of verifying the proof. For ZK proofs, verification compute is cheap and proof production is expensive. For audit-style verification, proof production is cheap (the records already exist) and verification compute is expensive (the auditor's time). The cost structure differs by verification technology.
3. Governance overhead. Trust and verification both require governance: who is allowed to be trusted, who is allowed to verify, what happens when trust is broken or verification fails. Governance overhead is often underweighted in cost models but is substantial in practice. Mature trust frameworks (NIST SP 800-207 zero trust architecture) explicitly include governance as a first-class cost [15].

4. Latency tradeoff. Trust is fast: trusted decisions can be made in milliseconds. Verification is slow: verified decisions take as long as the verification process. For latency-sensitive workloads, trust is often the only viable choice. For latency-tolerant workloads, verification can substitute for trust. The tradeoff is fundamental and workload-specific.

5. Reversibility. Trust-based decisions are often reversible: if a trusted party makes a bad decision, the decision can be reversed through remediation. Verification-based decisions are often irreversible: if verification confirms a decision, the decision is committed. The reversibility of the decision should shape the trust-verification mix.

Critical Caveats

Trust is not free — audits, insurance, reputation are all trust costs. The dominant framing in enterprise architecture treats trust as zero-cost (we trust our internal systems because they are internal) and verification as positive-cost (audits cost money). The correct framing treats both as positive-cost, with trust costs realized as remediation when trust fails. The remediation cost is often larger than the audit cost would have been.

Trust and verification are not always substitutes. Some flows need both — defense in depth. A high-stakes decision may be made by a trusted party (because latency requires it) and verified after the fact (because the stakes require it). The cost is trust cost plus verification cost, not the minimum. Defense in depth is more expensive than either alone, but it is sometimes the right answer.

Often, trust plus verification is cheaper than either alone. This counterintuitive result holds when trust reduces the scope of verification (we only verify anomalies) and verification reduces the cost of trust (we trust because we know we will check). The architecture decision is not always trust-or-verify; it is sometimes trust-and-verify-where-it-matters.

Counterarguments & Limitations

The trust cost formula depends on probability of betrayal, which is rarely measurable. Internal employees, long-term vendors, and trusted systems have betrayal probabilities that depend on context, incentive, and culture. The formula is correct in form but hard to populate with numbers. Architecture decisions based on the formula should acknowledge the uncertainty in the betrayal probability estimate.

Verification costs are also hard to measure. The direct cost of running a verifier is easy to measure; the indirect cost of maintaining the verification infrastructure, training the verification team, and responding to verification failures is harder. Total verification cost is typically 2-5x the direct verification cost.

The framework assumes rational actors. Trust and verification decisions are made by humans (or by systems designed by humans) under organizational pressures. Decisions that look irrational in the framework may be rational given the organization's incentive structure. The framework is most useful as a diagnostic to surface irrationality, not as a prescription to eliminate it.

Some flows require both trust and verification, which complicates the substitution logic. The framework's substitution assumption breaks when defense in depth is required. For high-stakes decisions where both trust and verification are mandatory, the cost is additive, and the architecture question shifts from “trust

or verify” to “how to design the verification layer to minimize its marginal cost given that trust is also present.”

Finally, the framework does not capture the cost of verification failure. When verification incorrectly flags a correct decision, the cost is the disruption caused by the false positive. When verification incorrectly accepts an incorrect decision, the cost is the realized loss. The framework assumes verification is perfectly accurate, which it never is. A more complete model would include verification accuracy as a parameter, with its own cost-benefit tradeoff.

A DECISION FRAMEWORK FOR EXECUTIVES, AND THE CONSULTING LEAD GENERATOR THAT THIS SERIES IS BUILT AROUND

Episode 05 — Build vs Buy vs Rollup

***Thesis.** The build-vs-buy decision is the most common architecture decision in enterprise technology, and it has a third option that is rarely considered: rollup. The rollup option — shared settlement, execution autonomy, proof overhead — is sometimes the right answer, and the framework we provide lets a technical leader evaluate it alongside the more familiar options.*

Thesis

Build, buy, and rollup are three architectural patterns for acquiring a capability. Build means implementing internally, with full control and full cost. Buy means purchasing a vendor solution, with fast time-to-deploy and recurring cost. Rollup means sharing a settlement layer with other participants while maintaining execution autonomy. Each pattern has a cost profile, a control profile, and a strategic profile that should be evaluated before committing.

Five Key Mechanisms

1. Build = full control + full cost. Building a capability internally gives the organization complete control over its features, performance, and roadmap. It also requires the organization to bear the full cost: engineering, operations, maintenance, and eventual replacement. Build is the right choice when the capability is strategic (differentiating), when the organization has the engineering capacity, and when the long-term cost of vendor dependence exceeds the long-term cost of internal operation [16].
2. Buy = fast + recurring cost + vendor lock-in. Buying a capability from a vendor gives fast time-to-deploy, access to vendor expertise, and operational predictability (the vendor's SLA). It also imposes recurring cost (licensing fees), vendor lock-in (switching costs are high), and strategic dependence (the vendor's roadmap shapes the buyer's). Buy is the right choice when the capability is non-strategic, when time-to-deploy is critical, and when the organization lacks the engineering capacity to build well.

3. Rollup = shared settlement + execution autonomy + proof overhead. The rollup option is the architectural innovation Ethereum introduced. In enterprise terms, rollup means participating in a shared settlement layer (an industry utility, a consortium chain, a regulated ledger) while maintaining execution autonomy (your own application services, your own data, your own policies). The cost is proof overhead: the organization must produce evidence of correct execution to participate in the shared layer. Rollup is the right choice when the capability is partially strategic (execution is strategic, settlement is not), when the organization can bear the proof overhead, and when the network effects of shared settlement are valuable.

4. Switching cost matrix. The three patterns have different switching costs. Build-to-buy is expensive (you have to dismantle internal infrastructure). Buy-to-build is expensive (you have to rebuild what the vendor provided). Rollup-to-either is moderate (the proof overhead can be repurposed, and the settlement layer can be exited). The switching cost shapes the strategic optionality of each choice and should be evaluated as part of the decision.

5. Strategic optionality. Each pattern forecloses future options differently. Build forecloses the option to switch vendors quickly. Buy forecloses the option to customize deeply. Rollup forecloses the option to settle independently. Real options analysis [17] provides the framework to value these foreclosed options and to include the option value in the decision.

Figure 2.1 — Build / Buy / Rollup Decision Tree

The decision turns on four axes: latency sensitivity, strategic importance, regulatory burden, and ecosystem availability. Most capabilities sit on a spectrum, not at a vertex.

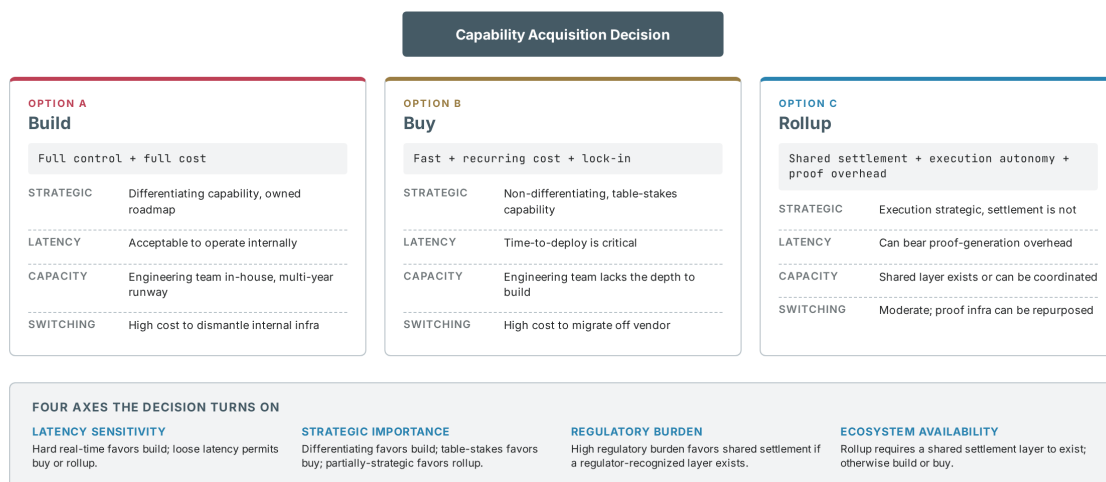


Figure 2.1 — Build / Buy / Rollup decision tree. The decision turns on four axes: latency sensitivity, strategic importance, regulatory burden, and ecosystem availability. Most enterprise capabilities sit on a spectrum rather than at a vertex.

Critical Caveats

Every choice forecloses future options. The build-vs-buy-vs-rollup decision is not just about current cost; it is about the cost of changing the decision later. Build decisions are sticky: once an organization has invested in internal infrastructure, the switching cost to a vendor or to a rollup pattern is high. The same applies in the other directions. Architecture decisions should be evaluated on a multi-year horizon, not

just on year-one cost.

The framework assumes rational buyers. In practice, vendor relationships, organizational politics, and sunk costs dominate enterprise architecture decisions. The framework's value is in making the rational case visible, not in overriding the non-rational factors. A technical leader who uses the framework to document the rational case and then to negotiate with non-rational factors will make better decisions than one who ignores either.

Rollup is a novel option and is unfamiliar to most enterprise buyers. The unfamiliarity is itself a cost (education, vendor selection, contract negotiation) and a risk (the pattern is less battle-tested than build or buy). Enterprises evaluating rollup should expect longer decision cycles and should budget for exploratory work before commitment.

Counterarguments & Limitations

The framework is a starting point, not a substitute for judgment. The three patterns and the five mechanisms are abstractions; real architecture decisions involve dozens of specific factors that the framework cannot capture. The framework is useful as a structure for organizing the analysis, not as an answer generator.

Vendor relationships and sunk costs dominate in practice. Many enterprise architecture decisions are made on the basis of existing vendor relationships, internal politics, and the cost of changing the status quo. The framework does not capture these factors, and a decision that is rational by the framework may be irrational by the organization's actual decision-making process.

The rollup option depends on a shared settlement layer existing. For most enterprise capabilities, no such layer exists today. The rollup pattern is aspirational for most use cases — a pattern that could be applied if the shared infrastructure existed, rather than a pattern that can be applied today. Enterprises that want to use the rollup pattern may need to invest in building the shared layer first, which changes the cost-benefit calculation.

Real options analysis is hard to apply rigorously. The framework depends on valuing foreclosed options, which requires estimating future states of the world that are inherently uncertain. The academic literature on real options is rich, but its application to specific enterprise decisions is more art than science. The framework should be used to surface option value, not to produce a precise number.

Finally, the framework is a lead generator for the consulting engagement that this document is designed to support. The honest version of this caveat is that the framework's value is realized through its application to a specific organization's specific decisions, and that application is what the consulting engagement provides. The blueprint depth of this episode is appropriate for an executive workshop, not for a self-service decision. The decision framework is the start of a conversation, not the end of one.

WHAT BLOCKCHAIN TAUGHT US ABOUT REAL-WORLD INFRASTRUCTURE. EACH EPISODE TRANSLATES A PATTERN TO A SPECIFIC INDUSTRIAL DOMAIN.

Series 3 — Crypto Architecture → Industrial Architecture

Thesis. *Most blockchain analysis stays inside blockchain. This series translates blockchain architecture lessons into the operational language of factories, logistics, mobility, supply chains, and industrial compliance — the domains where the patterns could actually pay off.*

Series 3 is the differentiation series. Series 1 transferred the architectural patterns in the abstract. Series 2 transferred the economic reasoning. Series 3 translates both into the operational language of specific industrial domains, because architecture patterns that cannot be expressed in a domain's vocabulary cannot be deployed in that domain.

The five episodes are paired: each takes one pattern from Series 1 and applies it to one industrial domain. Factories get the layering model. Logistics gets the rollup pattern. Mobility gets the sequencer. Supply chains get data availability. Industrial compliance gets ZK. The mapping is not exhaustive — each pattern applies to multiple domains, and each domain can use multiple patterns — but the specific pairings are the ones we think are most commercially relevant in the next 24 months.

GCC relevance is explicit, particularly in Episode 05. The data sovereignty, regulatory, and infrastructure-investment dynamics of the Gulf states make several of these patterns especially well-suited to the region's industrial base, and we name that fit where it exists.

FACTORY EDGE DEVICES EXECUTE LOCALLY; THE CENTRAL SYSTEM SETTLES IMPORTANT STATE. THE ROLLUP ANALOGY FOR MANUFACTURING.

Episode 01 — What Ethereum Teaches Us About Factories

Thesis. *A factory is, in architectural terms, a layered system with a slow, conservative core (the master production record, the calibration chain, the regulatory audit trail) and a fast, specialized periphery (PLC logic, edge analytics, operator interfaces). The L1/L2 pattern is a precise description of this layering, and ISA-95 — the manufacturing interoperability standard — already encodes most of it.*

Thesis

The architectural insight from Ethereum that applies to factories is not the blockchain. It is the disciplined separation of the trust root (the master production record, the calibration chain, the regulatory audit trail) from the execution layer (PLC logic, edge analytics, operator interfaces). This separation already exists in ISA-95, the manufacturing interoperability standard. What Ethereum adds is the publication-layer and verification-strategy discipline that ISA-95 leaves underspecified.

Five Key Mechanisms

1. Edge execution. PLCs and edge analytics devices execute control logic locally, at machine speed. The execution must be local because round-trip latency to a central system would be unacceptable for real-time control. Ethereum L2 rollups execute transactions locally for the same reason: locality preserves the latency property that the workload requires.
2. Periodic settlement. Important state changes — production counts, quality checks, equipment status transitions — are batched and committed to the central system periodically (every few seconds to every few minutes, depending on the application). This is the publication layer in Ethereum terms. The batching amortizes the central system's commitment cost across many edge events.
3. Data availability at the PLC level. The data needed to verify an edge decision (sensor readings, control outputs, operator interventions) must be available to central verification. In current factory architectures, this data is often stored in vendor-specific historian databases with limited accessibility. The DA-layer pattern suggests a dedicated, append-only evidence store that is independent of any single vendor's historian.
4. Verifiability for compliance. Regulatory regimes (FDA 21 CFR Part 11, ISO 13485 for medical devices, ISO 9001 for quality management) require audit trails for production decisions. The current state of factory audit is patchy: data is often incomplete, modifications are detectable only through database forensics, and the chain of custody between edge event and audit record is often broken. The DA-layer pattern, applied to factory evidence, would make compliance audit a query against a hash-committed store rather than a forensic investigation.
5. ISA-95 alignment. ISA-95 [1] defines levels 0-4 for manufacturing: level 0 is the physical process, level 1 is sensing and manipulation, level 2 is monitoring and supervisory, level 3 is manufacturing operations management, level 4 is business planning. The L1/L2 pattern maps cleanly: levels 0-2 are the periphery (execution), levels 3-4 are the core (settlement). What the L1/L2 pattern adds is the explicit interface discipline between levels 2 and 3, which ISA-95 leaves to vendor implementation.

Critical Caveats

Factory networks are not adversarial. The threat model for a factory floor is operator error, equipment failure, and the occasional insider threat, not the rational adversaries that Ethereum's cryptographic machinery is designed to resist. Applying Ethereum-grade cryptography to factory evidence is over-engineering for the actual threat model. The DA-layer pattern's value is in the discipline (append-only, hash-committed, verifier-accessible), not in the cryptography.

Determinism is more important than verifiability in OT. Operational technology values predictability above almost everything else. A verification layer that adds latency to the control loop is unacceptable, regardless of its audit value. The pattern must be applied to the audit path, not to the control path — the control loop stays local, the audit layer runs asynchronously.

Vendor lock-in is a real concern. Factory equipment has long lifecycles (20+ years) and is often locked to vendor-specific protocols and data formats. A DA-layer that requires vendor cooperation to extract evidence may not be feasible for legacy equipment. The pattern applies cleanly to new deployments and to brownfield deployments where evidence can be captured at the supervisory layer (level 2-3) without vendor cooperation at the device layer (level 0-1).

Counterarguments & Limitations

Factory floors lack Ethereum's economic security model. Ethereum's trust root is secured by billions of dollars of stake. A factory's trust root is secured by physical access controls, organizational policy, and regulatory audit. These are real but weaker. The DA-layer pattern does not give the factory the strength of L1 finality; it just makes explicit where the weaker factory-grade settlement guarantee lives.

ISA-95 already defines layering — adding a blockchain mental model may add cognitive overhead without operational benefit. Factory engineers trained on ISA-95 do not need a new vocabulary for layering; they need better tools for the interfaces between layers. The L1/L2 pattern's value is in the interface design discipline (publication layer, verification strategy), not in the layering concept itself. The selling point to a factory engineer is “better interfaces,” not “new layers.”

The pattern assumes greenfield deployment or brownfield supervisory access. For deeply legacy equipment with no supervisory-layer data extraction capability, the pattern is not applicable. The honest scoping is that the pattern applies to factory equipment installed in the last 15 years, which is roughly 50-70% of the installed base in developed markets and 30-50% in emerging markets.

Regulatory acceptance of DA-layer-stored evidence is not automatic. FDA 21 CFR Part 11, for example, has specific requirements for electronic records that include human-readable display and the ability to generate paper copies. A DA-layer that stores only cryptographic commitments may not satisfy these requirements without an additional presentation layer. The pattern needs to be paired with a regulatory-compliance layer that translates commitments into auditor-readable form.

Finally, the operational maturity of most factory IT organizations is not ready for a DA-layer discipline. Implementing the pattern requires cooperation between OT (which owns the equipment and the data), IT (which owns the infrastructure), and compliance (which owns the audit relationship). The organizational alignment is often the binding constraint, not the technology. The pattern's value is unlocked only when the organizational alignment exists.

DON'T TRANSMIT EVERY EVENT INDIVIDUALLY. AGGREGATE, COMPRESS EVIDENCE, SETTLE PERIODICALLY.

Episode 02 — What Rollups Teach Us About Logistics

***Thesis.** Logistics systems transmit too much data, too frequently, to central systems that cannot meaningfully process it. The rollup pattern — batch locally, compress, settle periodically — is the architectural answer, and logistics already practices a weak version of it through advance shipping notices and EDI. The pattern's value is in making the discipline explicit and verifiable.*

Thesis

Modern logistics generates more event data than its central systems can usefully consume. Container tracking, vehicle telemetry, warehouse scan events, port handoff events — the volume is too high to be processed in real time and too valuable to be discarded. The rollup pattern, applied to logistics, replaces the transmit-every-event pattern with a batch-and-settle pattern that preserves evidence while reducing bandwidth and central processing load.

Five Key Mechanisms

1. Event batching. Instead of transmitting every container scan or vehicle telemetry point to a central system, edge devices batch events locally and transmit batches periodically. The batching window depends on the application: real-time tracking may use 30-second batches; regulatory compliance may use daily batches. The batching reduces bandwidth by 1-3 orders of magnitude depending on the event pattern.
2. State compression. Within each batch, redundant events are compressed. A container that is scanned 50 times in a warehouse may produce one state change (“in warehouse, ready for shipment”) plus 49 contextual events that are summarized rather than individually transmitted. The compression discipline comes from the rollup pattern's state-root approach: transmit the resulting state, plus the evidence needed to verify the state, not the full event stream.
3. Periodic settlement. Batches are committed to a central system periodically, where they are settled into canonical state. The settlement frequency depends on the application: customs documentation may settle daily, billing may settle per-shipment, regulatory reporting may settle monthly. Each settlement produces a hash-committed record that downstream systems (billing, audit, compliance) reference.
4. Evidence availability. The full event stream underlying each batch is retained in an availability layer for the duration of the regulatory retention period. Auditors can query the availability layer to verify any specific event that contributed to a settled state. The evidence layer is the part of the pattern that distinguishes it from simple batch processing: the evidence is preserved, hash-committed, and available

for verification.

5. Chain of custody. Each batch's evidence records form a chain of custody from the originating edge device to the settled central state. The chain is verifiable: each link in the chain is hash-committed and signature-attested. The chain-of-custody property is what makes the pattern useful for regulated logistics (pharmaceuticals, high-value goods, dangerous goods) where custody disputes are common.

Critical Caveats

Logistics already does this through ASN/EDI — what is new? The honest answer is that the pattern is an extension and strengthening of existing ASN/EDI practice, not a fundamentally new approach. What is new is the hash-committed, verifier-accessible evidence layer that makes the chain of custody cryptographically verifiable rather than merely organizationally auditable. For most logistics applications, the cryptographic strengthening is not worth the additional infrastructure cost. For high-value or regulated goods, it may be.

Logistics partners are not adversarial in the same way Ethereum participants are. The threat model for logistics evidence is partner error, system discrepancy, and the occasional fraud, not the rational-adversary model that Ethereum's cryptography is designed to resist. Over-engineering the evidence layer for the actual threat model is a real cost without proportional benefit.

Legal frameworks already exist for shipment disputes. The Hague-Visby rules, the Rotterdam Rules, and national transport law provide a comprehensive framework for liability and dispute resolution in logistics. A cryptographic evidence layer does not replace these frameworks; it provides additional evidence that may be admissible within them. The incremental value over the existing legal framework is real but bounded.

Counterarguments & Limitations

Existing frameworks cover most use cases. ASN/EDI and the various logistics standards (GS1 EPCIS [2], ISO 28000 supply chain security [3]) cover the majority of logistics evidence requirements. The rollup pattern's value is at the margin — high-value goods, regulated goods, multi-party disputes — not at the center. For most shipments, the existing framework is sufficient, and the rollup pattern adds cost without proportional benefit.

Blockchain-style settlement adds cost without proportional risk reduction in most logistics flows. The risk that the pattern reduces — evidence manipulation, custody disputes, partner disputes — is rare in absolute terms and is already managed through insurance, contractual remedies, and the existing legal framework. The pattern's value is concentrated in flows where these existing mechanisms are insufficient, which is a small percentage of total logistics volume.

Multi-party logistics networks are hard to coordinate. The pattern assumes that all parties in a logistics flow participate in a shared evidence layer. In practice, logistics flows involve multiple independent parties (shipper, carrier, port operator, customs, consignee) with different systems, different standards, and different incentives. Achieving shared evidence infrastructure across this multiplicity is a coordination problem, not a technical problem.

The pattern's economics depend on transaction volume that most logistics flows do not have. The amortization that makes Ethereum rollups economic requires high transaction rates. Most logistics flows have low transaction rates per shipment (10-50 events per shipment, with thousands of shipments per shipper per day). The amortization is sufficient for high-volume shippers but marginal for low-volume ones.

Finally, regulatory acceptance of cryptographic evidence in logistics disputes is uneven. Some jurisdictions (notably Singapore, the UK, and increasingly the UAE) have begun to acknowledge cryptographic evidence in commercial disputes. Others have not. Enterprises adopting the pattern for cross-border logistics should expect a multi-year process of regulatory engagement in jurisdictions where the pattern's evidence is novel.

THOUSANDS OF VEHICLES GENERATE EVENTS. SOMEONE MUST DETERMINE PRIORITY, ROUTING, AUTHORIZATION, AND SETTLEMENT.

Episode 03 — What Sequencers Teach Us About Mobility

***Thesis.** Mobility systems — ride-sharing, fleet management, autonomous vehicle coordination, traffic management — have a sequencer problem at their core. The L2 sequencer pattern is the most-studied instance of the problem, and the Enterprise Decision Sequencer it suggests is a missing primitive in most mobility architectures.*

Thesis

Mobility systems with many concurrent vehicles and many concurrent events need an ordering policy for intersection precedence, route allocation, charging station access, and emergency vehicle priority. The L2 sequencer pattern, stolen faithfully, provides the architectural primitive. The translation has hard constraints: mobility safety cannot tolerate the latency of Ethereum's challenge windows, and the threat model is different. But the ordering problem is the same.

Five Key Mechanisms

1. Priority ordering. Vehicles compete for limited resources: intersection access, lane space, charging stations, parking spots. The sequencer's first function is to order these competing requests according to a priority policy. Emergency vehicles preempt ordinary vehicles; public transit preempts private vehicles in dedicated lanes; paying customers may preempt non-paying customers in commercial applications. The ordering policy is the architectural decision.

2. Routing arbitration. Sequencer decisions shape route recommendations. A vehicle that requests routing may receive a route that is suboptimal for itself but optimal for system-wide flow. The sequencer's

routing arbitration determines which vehicles get optimal routes and which get suboptimal routes. In Ethereum L2 terms, this is sequencer MEV: the value of ordering is captured by the sequencer and redistributed (or not) according to the protocol's rules.

3. Authorization sequencing. Vehicles must be authorized to perform certain actions: enter a restricted zone, use a toll road, charge at a specific station. The sequencer's authorization function determines which vehicles are permitted which actions, in which order. The authorization policy is typically regulatory (vehicles must meet emission standards to enter low-emission zones) and commercial (vehicles must have valid subscriptions to use toll roads).

4. Settlement at trip end. Each trip produces a settlement event: the route taken, the resources consumed (fuel, electricity, road usage), the charges incurred, the time elapsed. The settlement event is committed to the canonical ledger (the billing system, the fleet management system, the regulatory reporting system). The sequencer's settlement function determines how often and how completely trip state is committed.

5. ISO 21434 alignment. ISO 21434 [4] is the road vehicle cybersecurity engineering standard. It specifies requirements for cybersecurity in road vehicle systems, including requirements for logging, monitoring, and incident response. The sequencer's evidence-emission function aligns with ISO 21434's logging requirements and provides a structured record of vehicle decisions for regulatory audit.

Critical Caveats

Mobility has hard latency requirements that Ethereum does not. Vehicle safety decisions must be made in milliseconds; Ethereum L2 sequencers operate in seconds to minutes. The mobility sequencer cannot use Ethereum's challenge-window approach to verification — the challenge window must be near-zero, which means validity proofs (instant verification) rather than optimistic verification (challenge-after-the-fact). This is a hard constraint that shapes the entire architecture.

Vehicle autonomy requires sub-ms determinism, not cryptographic settlement. The vehicle's local control loop (perception, planning, control) operates at 10-100 Hz and cannot wait for any external sequencing decision. The sequencer pattern applies to fleet-level decisions (intersection precedence, route allocation) not to vehicle-local control. Conflating the two is a category error.

MEV has no clean mobility analog. The economic value of vehicle ordering is real (priority at intersections, optimal routing, charging station access) but is not as cleanly monetizable as financial MEV. The redistribution mechanisms Ethereum has developed (MEV auctions, proposer-builder separation) do not have direct mobility analogs. The mobility sequencer's economics are shaped by regulatory and commercial structures rather than by open markets for ordering.

Counterarguments & Limitations

Mobility safety cannot tolerate 7-day fraud-proof windows. The optimistic-rollup pattern is fundamentally incompatible with safety-critical mobility applications. The mobility sequencer must use either synchronous verification (too slow) or validity-proof verification (currently expensive for vehicle-grade computation). The technical path to validity-proof mobility sequencing exists but is not production-ready, and may not be for several years.

Vehicle autonomy requires sub-ms determinism, not cryptographic settlement. The sequencer pattern applies to fleet-level decisions, not to vehicle-local decisions. Conflating the two produces architectures that are unsafe for vehicle control and unnecessarily heavy for fleet coordination. The pattern must be scoped carefully to the fleet-coordination layer.

MEV has no mobility analog that justifies the machinery. The MEV literature is rich and valuable as design vocabulary, but the specific mechanisms (auctions, proposer-builder separation, encrypted mempools) do not have mobility counterparts. Importing the vocabulary without the mechanisms produces architecture that talks about MEV without capturing any value.

Regulatory frameworks for autonomous vehicle sequencing are still emerging. The NHTSA in the US, the UNECE WP.29 internationally, and various national regulators have begun to develop frameworks for autonomous vehicle operation, but these frameworks do not yet have clear positions on multi-vehicle sequencing. Enterprises adopting the pattern should expect regulatory engagement to be a multi-year process.

Finally, the political economy of mobility sequencing — who controls intersection precedence, who captures the value of priority access, who pays for the consequences of sequencing decisions — is a public policy question as much as an architecture question. The sequencer pattern makes the political economy visible; it does not resolve it. Cities, regulators, and mobility operators will need to develop answers that the pattern can then implement.

“DELIVERED” ON A DASHBOARD IS NOT TRUSTWORTHY WITHOUT EVIDENCE.

Episode 04 — What Data Availability Teaches Us About Supply Chains

***Thesis.** Supply chain visibility systems — the dashboards that show shipment status, inventory levels, and production progress — are only as trustworthy as the evidence behind them. The data availability pattern, applied to supply chain evidence, transforms dashboard status from an assertion into a verifiable claim.*

Thesis

A supply chain dashboard that shows “shipment delivered” is making a claim. The claim is trustworthy only if the evidence behind it — the GPS reading, the scan event, the signature, the timestamp — is available, durable, and complete. Most supply chain visibility systems make claims without preserving evidence; the data availability pattern is the architectural fix.

Five Key Mechanisms

1. Evidence records. Every status claim in a supply chain visibility system is backed by an evidence record. The record contains the status claim, the source (which system produced the claim), the evidence (sensor data, scan events, signatures), the timestamp, and the context (location, environmental conditions, operator identity). The record is the unit of verifiability.
2. Independent verification. The evidence layer is independently queryable by verifiers (auditors, regulators, downstream systems). Verifiers do not need to trust the visibility system's operator; they query the evidence layer directly. This is the property that distinguishes the pattern from vendor-controlled dashboards.
3. Audit trails. Each evidence record is part of a chain that extends from raw sensor data through processing to the dashboard claim. The chain is verifiable: each link is hash-committed and signature-attested. Auditors can trace any dashboard claim back to its raw evidence without trusting intermediate processing.
4. Multi-party attestation. High-stakes supply chain events (customs clearance, dangerous goods handoff, pharmaceutical chain of custody) involve multiple parties. The pattern supports multi-party attestation: each party signs the evidence record, and the record is not considered complete until all required signatures are present. Multi-party attestation is what makes the pattern useful for regulated supply chains.
5. Cold-chain and specialized extensions. Temperature-sensitive supply chains (pharmaceuticals, food) require continuous temperature monitoring. The pattern extends naturally: each temperature reading is an evidence record, each temperature excursion is a flag that may invalidate downstream claims. The cold-chain extension demonstrates the pattern's composability: the same evidence-record discipline that supports shipment status supports temperature monitoring, with different schemas but the same architectural primitive.

Critical Caveats

Supply chain audit frameworks already exist — ISO 28000, GS1 EPCIS, the WTO Trade Facilitation Agreement. These frameworks cover most of the ground that the DA-layer pattern covers, with decades of implementation experience. The DA-layer pattern's value is in the cryptographic strengthening of these frameworks, not in replacing them. Enterprises adopting the pattern should expect to integrate with existing frameworks rather than displace them.

Most supply chain evidence is already captured by someone — the question is access and verifiability, not capture. The pattern's value is in making existing evidence accessible to verifiers in a structured, hash-committed form. The investment is in the access layer and the commitment discipline, not in new sensing infrastructure.

Cross-organizational data sharing is a hard problem that the pattern does not solve. The pattern assumes that evidence records can be shared across organizational boundaries, but commercial sensitivities, regulatory restrictions, and competitive dynamics often prevent sharing. The pattern's value is realized only when the organizational alignment exists to permit sharing.

Counterarguments & Limitations

Existing frameworks cover most use cases. ISO 28000, GS1 EPCIS, and the various supply chain audit frameworks that have developed over the past 30 years cover the vast majority of supply chain evidence requirements. The DA-layer pattern adds cryptographic verifiability at the margin, for high-stakes or disputed cases. For most supply chain flows, the existing frameworks are sufficient.

Blockchain adds cost when the audit framework is the bottleneck, which is rare. The audit framework is rarely the binding constraint in supply chain disputes; commercial relationships, contractual remedies, and insurance are usually the binding constraints. Adding cryptographic evidence infrastructure does not address these binding constraints.

Multi-party evidence sharing requires organizational alignment that is often absent. Supply chains involve multiple independent organizations with different systems, different incentives, and different regulatory environments. Achieving shared evidence infrastructure across this multiplicity is a coordination problem that the pattern does not solve.

The pattern assumes that evidence records are complete in the sense that they capture everything needed for verification. In practice, supply chain evidence is often incomplete: sensor gaps, manual interventions, and contextual data that is not captured. A pattern that stores incomplete evidence records is worse than no pattern, because it provides false assurance.

Finally, regulatory acceptance of cryptographic supply chain evidence is uneven. Some regulators (notably the US FDA for pharmaceutical supply chains, and the EU for various food traceability regimes) have begun to acknowledge cryptographic evidence. Others have not. Enterprises adopting the pattern for cross-border supply chains should expect regulatory engagement to be a multi-year process in jurisdictions where the pattern's evidence is novel.

PROVE THAT A REQUIREMENT WAS SATISFIED, WITHOUT EXPOSING SENSITIVE DATA. GCC-RELEVANT.

Episode 05 — What ZK Teaches Us About Industrial Compliance

***Thesis.** Industrial compliance often requires proving that a requirement was satisfied without exposing the underlying sensitive data. ZK proofs are the cryptographic primitive for exactly this case, and the GCC's data sovereignty regime makes the pattern especially commercially relevant in the region.*

Thesis

Industrial compliance — environmental emissions reporting, financial controls, safety certifications, data residency — often requires organizations to prove that a requirement was met without exposing the underlying data. The current state of practice is to provide selective disclosure: a regulator reviews selected records under a non-disclosure agreement. ZK proofs offer a stronger alternative: prove the

requirement was satisfied, cryptographically, without exposing any underlying data. The pattern is commercially relevant where data sovereignty or competitive sensitivity makes selective disclosure insufficient.

Five Key Mechanisms

1. **Proof of compliance.** A ZK proof attests that a compliance requirement was satisfied — emissions below threshold, financial controls properly applied, safety certifications valid — without exposing the underlying production data. The proof is cryptographically sound: it cannot be forged, and it reveals nothing beyond the compliance claim.
2. **Selective disclosure.** For cases where full ZK is not yet practical, the pattern supports selective disclosure: the organization reveals a subset of evidence that supports the compliance claim, with cryptographic commitments to the remaining evidence. Selective disclosure is weaker than full ZK but stronger than traditional audit, and it is implementable today with current technology.
3. **Audit reduction.** The pattern reduces the cost of regulatory audit by replacing detailed record review with proof verification. A regulator who can verify a compliance proof in seconds rather than reviewing thousands of records in days has lower audit cost and higher audit coverage. The economics are favorable for both sides, which is the strongest argument for regulatory adoption.
4. **Cross-jurisdictional portability.** A compliance proof generated under one regulatory regime can, in principle, be recognized by another regime without re-audit. This is particularly valuable for multinational operations that currently face duplicate audit requirements across jurisdictions. The portability is currently aspirational — regulators have not yet developed mutual recognition frameworks for cryptographic compliance proofs — but it is the most valuable long-term property of the pattern.
5. **GCC relevance for data sovereignty.** The Gulf states' data sovereignty regimes — Saudi Arabia's PDPL, the UAE's Data Office framework, Qatar's data protection law — impose restrictions on cross-border data transfer that create natural demand for compliance proofs that do not require data transfer. A ZK proof of compliance, generated on data that remains in-country, satisfies the data sovereignty requirement while providing cross-border verifiability. This is a genuine commercial fit, not a marketing claim.

Critical Caveats

ZK proof infrastructure is immature. The tooling for generating ZK compliance proofs is currently at the research-and-development stage, with limited production deployment. Enterprises adopting the pattern should expect significant engineering investment in circuit development and should plan for the technology to evolve over the next 2-3 years.

Regulatory acceptance is not automatic. Regulators have not yet developed consistent guidance on the use of ZK proofs for compliance. Some regulators (the UAE Data Office, Saudi SDAIA) have signaled openness to cryptographic attestation; others have not. Enterprises adopting the pattern should expect a multi-year regulatory engagement process.

Cost of proof generation may exceed audit cost savings for most compliance flows. The current cost of generating ZK proofs for complex compliance claims is high enough that the pattern is economic only for high-stakes compliance (financial controls, pharmaceutical chain of custody, environmental emissions of high-emission industries). For routine compliance, traditional audit remains cheaper.

Counterarguments & Limitations

Regulators often require raw data access, not proofs. Many regulatory regimes grant regulators the right to inspect underlying records on demand, and a ZK proof does not satisfy this right. The pattern complements traditional regulatory access rather than replacing it; regulators retain their raw-data access rights, and ZK proofs provide additional, lower-cost verification for routine compliance.

ZK compliance is currently more promise than practice. The academic literature on ZK proofs is rich, and the tooling has matured significantly, but production deployment for industrial compliance is rare. The pattern's commercial value depends on production deployment at scale, which is 2-5 years away for most applications.

Cost of proof generation may exceed audit cost savings for most compliance flows. The crossover point at which ZK proofs are economic for compliance is workload-specific and depends on the relative cost of proof generation versus audit cost. For high-volume, low-stakes compliance flows, traditional audit will remain cheaper for the foreseeable future. For low-volume, high-stakes compliance flows, the crossover may already have been crossed.

Circuit security and audit infrastructure for ZK are immature. As discussed in Series 2 Episode 03, ZK circuits have bugs and the audit infrastructure for circuits is less mature than for traditional code. Enterprises adopting ZK for compliance should expect to invest in circuit audit and should plan for circuit bugs.

Finally, the GCC relevance argument assumes the regulatory environment continues to evolve toward acceptance of cryptographic attestation. The current trajectory is favorable but not guaranteed. Enterprises making strategic bets on the pattern should monitor the regulatory trajectory closely and should diversify their compliance strategy rather than depending exclusively on ZK.

HOW THE THREE SERIES INTERLOCK AS A SINGLE COMMERCIAL FUNNEL

Cross-Series Synthesis

***Thesis.** Series 1 generates attention and transfers a mental model. Series 2 turns the attention into a structured decision the reader can apply. Series 3 translates the decision into the operational language of a specific industry. The flagship (Steal) is the intellectual property; ROI is the qualification mechanism; Industrial is the differentiation.*

The three series in this document are designed to interlock as a single commercial funnel, and the synthesis is the place where we make that explicit. We are not making a video to hope for views; we are making a body of work where each series transfers a specific capability that the next series requires.

Series 1 (Steal the Architecture) is the attention layer. Its job is to circulate widely, to be technically credible under peer review, and to transfer a mental model that a CTO or architect can use the next time they face a system design problem. The flagship status of Series 1 is not editorial preference; it is commercial structure. If Series 1 does not generate attention and transfer a model, the rest of the funnel is unreachable.

Series 2 (Architecture ROI) is the qualification layer. Its job is to take the attention generated by Series 1 and turn it into a structured decision a technical leader can apply to their own context. The build-vs-buy-vs-rollup framework in Series 2 Episode 05 is the consulting lead generator: it gives the reader enough framework to recognize that the decision is non-trivial, and it makes the case that the deeper analysis is what the consulting engagement provides.

Series 3 (Industrial Architecture) is the translation layer. Its job is to take the architectural and economic thinking of Series 1 and 2 and translate it into the operational language of a specific industry. A logistics CTO who reads Series 3 Episode 02 should be able to see themselves in the episode and recognize that the rollup pattern, applied to their ASN/EDI infrastructure, is non-trivial but potentially valuable. Without Series 3, Series 1 and 2 remain abstract; with Series 3, they become actionable.

The commercial funnel that the three series support is: free content (this document, the published episodes) transfers attention and mental model; a lead magnet (the Enterprise L2 Architecture Decision Matrix, not included in this document) transfers a structured question; a paid diagnostic (the Architecture Bottleneck Audit) transfers an actual answer for the reader's specific system; a high-ticket engagement (the Trust Infrastructure Architecture Sprint) transfers a design; and an enterprise implementation engagement transfers operational capability. Each stage of the funnel is fed by the work done at the previous stage, and the three series collectively are the foundation that makes the later stages credible.

We close the synthesis with a single editorial commitment. The funnel only works if the foundation is honest. If Series 1 overstates the strength of the analogy between Ethereum and enterprise architecture, the rest of the funnel collapses under technical review. We have therefore invested asymmetrically in the Counterarguments & Limitations sections throughout this document — not because we enjoy self-criticism, but because the commercial structure depends on the intellectual honesty. A reader who completes the three series should come away with a clear view of where the architecture pattern is powerful, where it is weak, and where the boundary between the two falls. That clarity is what earns the right to the consulting engagement. Without it, the document is marketing. With it, the document is architecture.

BIBLIOGRAPHY

Bibliography

Citations are numbered in order of first appearance, in IEEE style. Where a source is canonical (an EIP, a published paper, a standards document), the canonical version is cited. Where a source is industry documentation that evolves, the most recent publicly-available version at time of writing is cited.

- [1] Buterin, V., *Endgame*, vitalik.ca, 2021. <https://vitalik.ca/general/2021/12/06/endgame.html>
- [2] Ethereum Improvement Proposal 4844, *Proto-Danksharding: Blob-carrying transactions*, EIP repository, 2022. <https://eips.ethereum.org/EIPS/eip-4844>
- [3] Ethereum Improvement Proposal 1559, *Fee market change for ETH 1.0 chain*, EIP repository, 2019. <https://eips.ethereum.org/EIPS/eip-1559>
- [4] Buterin, V., *A rollup-centric ethereum roadmap*, ethereum.org, 2020. <https://ethereum.org/en/roadmap/rollup-centric/>
- [5] Buterin, V., *An Incomplete Guide to Rollups*, vitalik.ca, 2021. <https://vitalik.ca/general/2021/01/05/rollup.html>
- [6] Ethereum Foundation, *Rollup FAQ*, ethereum.org, 2023. <https://ethereum.org/en/developers/docs/scaling/rollups/>
- [7] Optimism Collective, *The Optimistic Rollup specification*, Optimism docs, 2024. <https://docs.optimism.io/>
- [8] Offchain Labs, *Arbitrum Nitro technical specification*, Arbitrum docs, 2024. <https://docs.arbitrum.io/>
- [9] Ben-Sasson, E., Bentov, I., Horesh, Y., Riabzev, M., *Scalable, transparent proofs of work (STARKs)*, IACR ePrint 2018/046, 2018. <https://eprint.iacr.org/2018/046>
- [10] Gabizon, A., Williamson, Z., Ciobotaru, O., *PLONK: Permutations over Lagrange-bases for Oecumenical Noninteractive arguments of Knowledge*, IACR ePrint 2019/953, 2019. <https://eprint.iacr.org/2019/953>
- [11] L2Beat, *L2Beat Risk Framework (Stage 1-3 classification)*, L2Beat.com, 2024. <https://medium.com/l2beat/introducing-stages-a-framework-to-evaluate-rollups-maturity-d290bb22befe>
- [12] Celestia Labs, *Modular blockchain design (whitepaper)*, Celestia.org, 2023. <https://celestia.org/learn/>
- [13] EigenLayer, *EigenLayer: The restaking primitive (whitepaper)*, EigenLayer docs, 2023. <https://docs.eigenlayer.xyz/>
- [14] Avail Project, *Avail data availability layer documentation*, Avail.project, 2024. <https://docs.avail.project/>
- [15] Kalodner, H., Goldfeder, S., Chen, X., et al., *Arbitrum: Scalable, private smart contracts (paper)*, USENIX Security, 2018. <https://arbitrum.io/papers/arbitrum.pdf>
- [16] Buterin, V., *How do ZK-Rollups work?*, ethereum.org, 2021. <https://ethereum.org/en/developers/docs/scaling/zk-rollups/>
- [17] Espresso Systems, *Decentralized sequencer design (technical report)*, Espresso Systems docs, 2024. <https://docs.espressosys.com/>
- [18] Astria, *Shared sequencer architecture*, Astria docs, 2024. <https://docs.astria.org/>
- [19] Flashbots, *MEV-Boost: PBS implementation for Ethereum proof-of-stake*, Flashbots docs, 2022. <https://boost.flashbots.net/>
- [20] Daian, P., Goldfeder, S., Kell, T., Li, Y., Zhao, X., et al., *Flash Boys 2.0: Frontrunning in Decentralized Exchanges, Miner Extractable Value, and Consensus Instability*, IEEE Symposium on Security and Privacy, 2020. <https://ieeexplore.ieee.org/document/9152675>

- [21] Roughgarden, T., *Transaction Fee Mechanism Design for the Ethereum Blockchain: An Economic Analysis of EIP-1559*, EC '20: Proc. 21st ACM Conference on Economics and Computation, 2020. <https://arxiv.org/abs/2012.00854>
- [22] Ethereum Improvement Proposal 4337, *Account Abstraction Using Alt Mempool*, EIP repository, 2021. <https://eips.ethereum.org/EIPS/eip-4337>
- [23] Arbitrum Foundation, *BoLD: Bounded Liquidity Dispute protocol*, Arbitrum research, 2024. <https://docs.arbitrum.io/bold/gentle-introduction>
- [24] Optimism Collective, *Fault proof specification (Cannon)*, Optimism docs, 2024. <https://docs.optimism.io/stack/protocol/fault-proofs/overview>
- [25] Polygon Labs, *Polygon PoS sidechain (whitepaper)*, Polygon docs, 2020. <https://polygon.technology/papers/>
- [26] Optimism Collective, *Bedrock: Rollup construction specification*, Optimism docs, 2023. <https://docs.optimism.io/stack/getting-started>
- [27] Ethereum Cat Herders, *Dencun network upgrade overview (EIP-4844 included)*, ethereum.org, 2024. <https://blog.ethereum.org/2024/02/27/dencun-mainnet-announcement>
- [28] L2Beat, *Stages framework for evaluating rollup maturity*, L2Beat.com, 2023. <https://l2beat.com/scaling/summary>
- [29] Yakovenko, A., *Solana: A new architecture for a high-performance blockchain (whitepaper)*, Solana.com, 2018. <https://solana.com/solana-whitepaper.pdf>
- [30] Buchwald, C., Layne, S., Williams, C., *Inter-Blockchain Communication protocol specification*, Cosmos Network, 2023. <https://github.com/cosmos/ibc>
- [31] Lamport, L., Shostak, R., Pease, M., *The Byzantine Generals Problem*, ACM Transactions on Programming Languages and Systems (TOPLAS), 1982. <https://dl.acm.org/doi/10.1145/357172.357176>
- [32] Lamport, L., *Time, clocks, and the ordering of events in a distributed system*, Communications of the ACM, 1978. <https://dl.acm.org/doi/10.1145/359545.359563>
- [33] Schneider, F. B., *Implementing fault-tolerant services using the state machine approach: a tutorial*, ACM Computing Surveys, 1990. <https://dl.acm.org/doi/10.1145/98163.98167>
- [34] Brewer, E. A., *Towards robust distributed systems (CAP conjecture, keynote)*, PODC '00 (Principles of Distributed Computing), 2000. <https://people.eecs.berkeley.edu/~brewer/cs262b-2004/PODC-keynote.pdf>
- [35] Gilbert, S., Lynch, N., *Brewer's conjecture and the feasibility of consistent, available, partition-tolerant web services (CAP proof)*, ACM SIGACT News, 2002. <https://groups.csail.mit.edu/tds/papers/lynch/sigact-2004.pdf>
- [36] Hohpe, G., Woolf, B., *Enterprise Integration Patterns: Designing, Building, and Deploying Messaging Solutions*, Addison-Wesley Professional, 2003. <https://www.enterpriseintegrationpatterns.com/>
- [37] Evans, E., *Domain-Driven Design: Tackling Complexity in the Heart of Software*, Addison-Wesley Professional, 2003. <https://www.domainlanguage.com/ddd/>
- [38] Vernon, V., *Implementing Domain-Driven Design*, Addison-Wesley Professional, 2013. https://www.dddcommunity.org/library/vernon_2011/
- [39] Fowler, M., *Patterns of Enterprise Application Architecture*, Addison-Wesley Professional, 2002. <https://martinfowler.com/books/eaa.html>
- [40] Rose, S., Borchert, O., Mitchell, S., Connelly, S., *Zero Trust Architecture (NIST SP 800-207)*, National Institute of Standards and Technology, 2020. <https://csrc.nist.gov/publications/detail/sp/800-207/final>
- [41] World Wide Web Consortium (W3C), *Verifiable Credentials Data Model 1.1*, W3C Recommendation, 2022. <https://www.w3.org/TR/vc-data-model/>
- [42] National Institute of Standards and Technology, *Artificial Intelligence Risk Management Framework (AI RMF 1.0)*, NIST AI 100-1, 2023. <https://www.nist.gov/itl/ai-risk-management-framework>

- [43] International Organization for Standardization, *ISO/IEC 27001:2022 Information security management systems*, ISO, 2022. <https://www.iso.org/standard/27001>
- [44] The Open Group, *TOGAF Standard, 10th Edition*, Open Group Standard, 2022. <https://www.opengroup.org/capabilities/togaf>
- [45] Hohpe, G., *The Architect Elevator: Connecting Technology and Business*, O'Reilly Media, 2024. <https://architectelevator.com/>
- [46] Beyer, B., Jones, C., Petoff, J., Murphy, N. R., *Site Reliability Engineering: How Google Runs Production Systems*, O'Reilly Media, 2016. <https://sre.google/sre-book/table-of-contents/>
- [47] Reed, P., Berg, N., *The Site Reliability Workbook: Practical Ways to Implement SRE*, O'Reilly Media, 2018. <https://sre.google/workbook/table-of-contents/>
- [48] Tyree, J., Ackerman, A., *Architecture decisions: Demystifying architecture*, IEEE Software, 2005. <https://ieeexplore.ieee.org/document/1429029>
- [49] Kreps, J., *I Heart Logs: Event Data, Stream Processing, and Data Integration*, O'Reilly Media, 2014. <https://www.oreilly.com/library/view/i-heart-logs/9781491909379/>
- [50] Deutsch, P., *The Eight Fallacies of Distributed Computing (originally the Seven Fallacies, expanded)*, Sun Microsystems, 1994. <https://blogs.oracle.com/developers/post/fallacies-of-distributed-computing>
- [51] Christensen, C. M., *The Innovator's Dilemma: When New Technologies Cause Great Firms to Fail*, Harper Business, 1997. <https://claytonchristensen.com/books/the-innovators-dilemma/>
- [52] Christensen, C. M., Raynor, M. E., *The Innovator's Solution: Creating and Sustaining Successful Growth*, Harvard Business Review Press, 2003. <https://claytonchristensen.com/books/the-innovators-solution/>
- [53] Dixit, A. K., Pindyck, R. S., *Investment Under Uncertainty*, Princeton University Press, 1994. <https://press.princeton.edu/books/hardcover/9780691034102/investment-under-uncertainty>
- [54] Amram, M., Kulatilaka, N., *Real Options: Managing Strategic Investment in an Uncertain World*, Harvard Business School Press, 1999. <https://www.hbs.edu/faculty/Pages/item.aspx?num=29041>
- [55] Forrester Research, *Total Economic Impact (TEI) Methodology*, Forrester, 2023. <https://www.forrester.com/what-it-means/tei/>
- [56] Cole, R. E., and the McKinsey team, *Three horizons of growth*, Harvard Business Review (McKinsey Quarterly reprint), 1999. <https://www.mckinsey.com/capabilities/strategy-and-corporate-finance/our-insights/three-horizons-for-growth>
- [57] Gartner Research, *Build versus buy: Framework for application delivery decisions*, Gartner, 2023. <https://www.gartner.com/en/documents/4253401>
- [58] L2Beat, *L2 fee benchmarks and economic analysis (continuously updated)*, L2Beat.com, 2024. <https://l2beat.com/scaling/economics>
- [59] Espresso Systems, *Sequencer economics: shared sequencing incentive analysis*, Espresso research, 2024. <https://blog.espressosys.com/>
- [60] Astria, *Shared sequencer incentive design*, Astria research, 2024. <https://blog.astria.org/>
- [61] Carta, S., Fedorov, M., Garimella, K., et al., *MEV-Cross-domain: The next frontier of Miner Extractable Value*, arXiv preprint, 2022. <https://arxiv.org/abs/2210.08563>
- [62] Buterin, V., *The dawn of decentralized finance*, vitalik.ca, 2021. <https://vitalik.ca/general/2021/12/19/endgame.html>
- [63] Daian, P., Goldfeder, S., *MEV auction design: Proposer-builder separation*, Flashbots research, 2021. <https://writings.flashbots.net/research/solving-mev/mev-auction-design/>
- [64] Easley, D., Lopez de Prado, M. M., O'Hara, M., *The coupling of AI and high-frequency trading: Maximally Extractable Value and the new market microstructure*, Journal of Financial Data Science, 2024. <https://doi.org/10.3905/jfds.2024.1.078>

- [65] Clements, R., Kazman, R., Bass, L., *Software Architecture in Practice (4th Edition, including ATAM)*, Addison-Wesley Professional, 2021.
<https://www.oreilly.com/library/view/software-architecture-in/9780136886449/>
- [66] Conway, M. E., *How do committees invent? (Conway's Law)*, Datamation magazine, 1968.
https://www.melconway.com/Home/Committees_Paper.html
- [67] Brooks, F. P., *The Mythical Man-Month: Essays on Software Engineering (20th Anniversary Edition)*, Addison-Wesley Professional, 1995.
<https://www.oreilly.com/library/view/the-mythical-man-month/0201835959/>
- [68] Simon, H. A., *The architecture of complexity (originally Proc. American Philosophical Society)*, reprinted in 'The Sciences of the Artificial', MIT Press, 1962.
<https://mitpress.mit.edu/9780262691918/the-sciences-of-the-artificial/>
- [69] Internet Engineering Task Force (IETF), *RFC 2119: Key words for use in RFCs to Indicate Requirement Levels*, IETF RFC, 1997. <https://www.rfc-editor.org/rfc/rfc2119>
- [70] Buterin, V., *Different types of layer 2s*, vitalik.ca, 2021. <https://vitalik.ca/general/2021/05/23/rollups.html>
- [71] Polygon Labs, *Polygon zkEVM: Technical overview (whitepaper)*, Polygon Labs, 2022.
<https://docs.polygon.technology/zkEVM/>
- [72] RISC Zero, *RISC Zero zkVM: Zero-knowledge proofs for arbitrary Rust code*, RISC Zero, 2023.
<https://www.risczero.com/>
- [73] Succinct Labs, *SPI: A performant, open-source zero-knowledge VM*, Succinct Labs, 2024. <https://succinct.xyz/>
- [74] Electric Coin Company, *Halo2: Recursive zk-SNARKs without trusted setup*, Electric Coin Company, 2022.
<https://zcash.github.io/halo2/>
- [75] Kothapalli, A., Setty, S., Tzialla, I., *Nova: Recursive zero-knowledge arguments from folding schemes*, IACR ePrint 2021/370, 2021. <https://eprint.iacr.org/2021/370>
- [76] International Organization for Standardization, *ISO 28000:2022 Security and resilience - Specification for supply chain security management systems*, ISO, 2022. <https://www.iso.org/standard/71554.html>
- [77] GS1, *EPC Information Services (EPCIS) standard*, GS1 Standards, 2023. <https://www.gs1.org/standards/epcis>
- [78] International Society of Automation, *ANSI/ISA-95: Enterprise-control system integration*, ISA, 2018.
<https://www.isa.org/standards-and-publications/isa-standards/isa-standards-committees/isa95>
- [79] International Organization for Standardization, *ISO 21434:2021 Road vehicles - Cybersecurity engineering*, ISO, 2021. <https://www.iso.org/standard/70918.html>
- [80] Saltzer, J. H., Reed, D. P., Clark, D. D., *End-to-end arguments in system design*, ACM Transactions on Computer Systems (TOCS), 1984. <https://dl.acm.org/doi/10.1145/357401.357402>